



Leader of Microcontroller Technology
Global Top Smart MCU Innovation Company

1-Shunt FOC Library

32-bit M-Series

Rev. 1.0.3
Nov. 08, 2022

*For additional information or inquiry, please contact ABOV Semiconductor
or visit its website at www.abov.co.kr.*

Copyright Information

The content of this document is subject to change without prior notice.

This document is intended to illustrate how to apply our product, and ABOV Semiconductor is not responsible for any copyright or patent infringement of third parties relating to the use of this product due to this document. This document does not contain any information on the proprietary rights of ABOV Semiconductor or of any other third party.

This product is intended to be used for general electronic devices, including office machines, communication devices, measuring devices, and household appliances.

Please consult ABOV Semiconductor if this product is used for high-performance, high-reliability, or safety-critical applications, such as nuclear energy control, spaceflight/aviation, traffic signals, combustion control, and various safety equipment. We do not hold responsible for any damages caused by products developed without consulting us in advance.

Copyright © 2019 ABOV Semiconductor Co., Ltd.

Revision History

Date	Version	Description
Jun. 28, 2019	1.0.0	Document created.
Oct. 8, 2019	1.0.1	Updated about aFDT
Apr. 19, 2021	1.0.2	Added A33M11x series
Nov. 8, 2022	1.0.3	Changed style and font

Contents

Chapter 1. Overview	9
1.1 Introduction	9
1.2 Architecture	10
1.3 Compatible MCUs	11
1.4 Getting Started	12
1.4.1 KEIL	12
1.4.2 IAR	16
Chapter 2. MDL (Motor Driver Layer)	19
2.1 Overview	19
2.2 Description of Functions	20
2.2.1 MPWMn_IRQHandler	20
2.2.2 MDL_getMPWMStatus	20
2.2.3 MDL_clearMPWMStatus	21
2.2.4 MDL_getADCShunt	21
2.2.5 MDL_checkFault	22
2.2.6 MDL_pwmON	22
2.2.7 MDL_pwmOFF	22
2.2.8 MDL_ATRInit	23
2.2.9 MDL_setATR	23
2.2.10 MDL_AllHalfPeriod	24
2.2.11 MDL_writePWMDuty	24
2.2.12 MDL_getADC	25
2.2.13 MDL_DMA_reset	25
2.2.14 MDL_ADC_init	25
2.2.15 MDL_MPWM_init	26
2.2.16 MDL_PCU_Init	26
Chapter 3. FAL (FOC Algorithm Layer)	27
3.1 Overview	27
3.2 Description of Functions	29
3.2.1 Main Functions of FAL	29
3.2.2 Calculation Functions	43
3.2.3 Configuration Functions	45
3.2.4 Read Functions	51
Chapter 4. MAL (Motor Application Layer)	54
4.1 Overview	54
4.2 Description of Functions	54
4.2.1 MAL_init	54
4.2.2 MAL_MPWM_Handler	54
4.2.3 MAL_mainloop	55

Chapter 5. aFDT (ABOV FOC Development Tool)	56
5.1 Overview	56
5.2 1-Shunt / 3-Shunt Selection	57
5.3 Parameter Configuration	58
5.4 Save/Load	59
5.4.1 Save	59
5.4.2 Load	60
5.5 Export	61
5.6 Parameter Description	63
5.6.1 Motor Parameter	63
5.6.2 Electrical Parameter	64
5.6.3 Align	65
5.6.4 Acceleration	66
5.6.5 Estimator	67
5.6.6 Speed Controller	68
5.6.7 Current Controller	69
5.6.8 Flux Weakening	70
5.6.9 1-Shunt	71
5.6.10 Software Overcurrent Level	72
5.6.11 System	72
5.7 Monitoring Function	73
5.7.1 Graph Area	74
5.7.2 Control Area	75

List of Figures

Figure 1.1 FOC Solution of ABOV	9
Figure 1.2 Layered architecture of ABOV library	10
Figure 1.3 Project folder.....	12
Figure 1.4 Project file	12
Figure 1.5 KEIL project options.....	13
Figure 1.6 Project options – Debug tab	13
Figure 1.7 Debugger configuration	14
Figure 1.8 Debugger configuration – Flash Download tab	14
Figure 1.9 Configuration of ABOV MCU Flash loader	15
Figure 1.10 Configuration of Flash loader upload path	15
Figure 1.11 IAR project folder	16
Figure 1.12 IAR project file	16
Figure 1.13 IAR project options	17
Figure 1.14 Designation of IAR linker configuration file (requires change of default installation path)	17
Figure 1.15 IAR project debugger configuration	18
Figure 1.16 IAR project CMSIS-DAP configuration	18
Figure 3.1 Block diagram of the 1-Shunt FOC function	27
Figure 3.2 Flow chart of the 1-Shunt FOC algorithm.....	28
Figure 3.3 Block diagram of the Sensorless Estimator function	29
Figure 3.4 Block diagram of the Speed Control function	30
Figure 3.5 Block diagram of the Current Control function	31
Figure 3.6 Block diagram of the SVPWM function	32
Figure 3.7 Block diagram of the Voltage Modulation function	33
Figure 3.8 Align configuration of aFDT	34
Figure 3.9 U-phase current waveform	34
Figure 3.10 Acceleration configuration of aFDT	35
Figure 3.11 U-phase current waveform	35
Figure 3.12 Block diagram of the Clarke Transformation function	36
Figure 3.13 Block diagram of the Inverse Clarke Transformation function.....	37
Figure 3.14 Block diagram of the Park Transformation function	38
Figure 3.15 Block diagram of the Inverse Park Transformation function.....	39
Figure 3.16 Block diagram of phase current calculation.....	41
Figure 3.17 Block diagram of the Set ADC Trigger function	42
Figure 3.18 Configuration of current controller bandwidth of aFDT	46
Figure 3.19 Configuration of Estimator of aFDT	48
Figure 5.1 Block diagram of aFDT communications.....	56
Figure 5.2 Selection page of FOC algorithms	57
Figure 5.3 Main page of the parameter generation	58
Figure 5.4 Popup for Save in aFDT	59
Figure 5.5 CSV file opened in Notepad	59
Figure 5.6 Popup for parameter generator loading	60
Figure 5.7 Popup for Export in aFDT <1>.....	61

Figure 5.8 Popup for Load in aFDT <2>	61
Figure 5.9 Header of a C file export.....	62
Figure 5.10 Motor parameters	63
Figure 5.11 Electrical parameters	64
Figure 5.12 Align parameters.....	65
Figure 5.13 Acceleration parameters.....	66
Figure 5.14 Estimator parameters	67
Figure 5.15 Speed controller parameters	68
Figure 5.16 Current controller parameters.....	69
Figure 5.17 Flux weakening parameters	70
Figure 5.18 1-Shunt parameters.....	71
Figure 5.19 Software overcurrent level parameters	72
Figure 5.20 System parameters	72
Figure 5.21 aFDT monitoring	73
Figure 5.22 Graph area.....	74
Figure 5.23 Graph area.....	75

List of Tables

Table 1.1 Layers of the ABOV 1-Shunt FOC Library	10
Table 1.2 ABOV M Series MCUs	11
Table 5.1 Items in the header of a C file export	62
Table 5.2 Motor Parameter	63
Table 5.3 Electrical Parameter	64
Table 5.4 Align.....	65
Table 5.5 Acceleration.....	66
Table 5.6 Estimator	67
Table 5.7 Speed Controller	68
Table 5.8 Current Controller.....	69
Table 5.9 Flux Weakening.....	70
Table 5.10 1-Shunt.....	71
Table 5.11 Software Overcurrent Level.....	72
Table 5.12 System	72
Table 5.13 Graph control buttons.....	74
Table 5.14 Graph display mode	75

Chapter 1. Overview

1.1 Introduction

The ABOV 1-Shunt FOC (Field Oriented Control) Library has been developed to operate a PMSM (Permanent Magnet Synchronous Motor) by ABOV MCU based on the FOC algorithm. It is used to operate compressors and fans of general household appliances, and to operate PMSMs in other applications. This library is designed for use in FOC platform boards, and requires modification when used in other types of boards.

The ABOV FOC solution includes a Windows GUI tool to allow easier modification of the ABOV 1-Shunt FOC Library and its internal parameters.

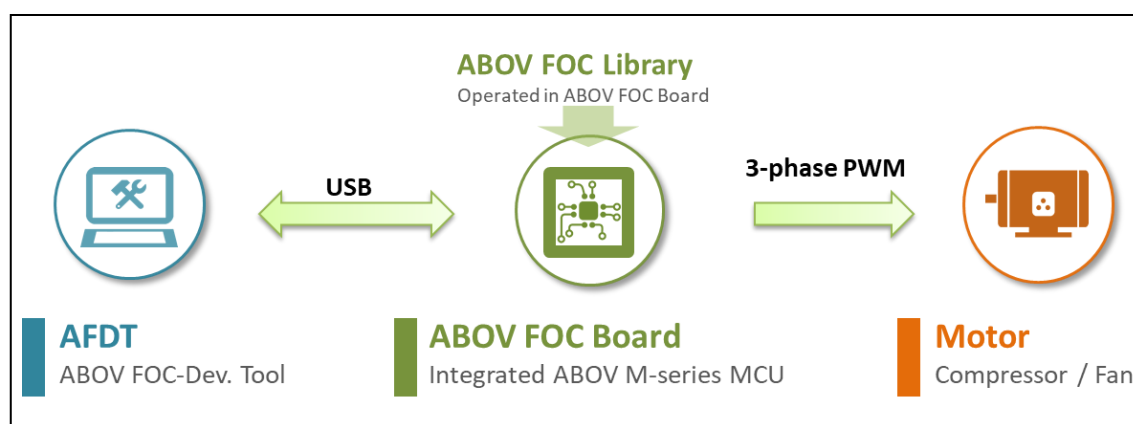


Figure 1.1 FOC Solution of ABOV

1.2 Architecture

The library is comprised of three layers.

Table 1.1 Layers of the ABOV 1-Shunt FOC Library

Layer	Description
Motor Application Layer	The motor application layer excluding the FOC core algorithm.
FOC Algorithm Layer (Locked)	The FOC core algorithm layer. Internal code is not accessible.
Motor Driver Layer	The MCU driver layer for peripheral controls.

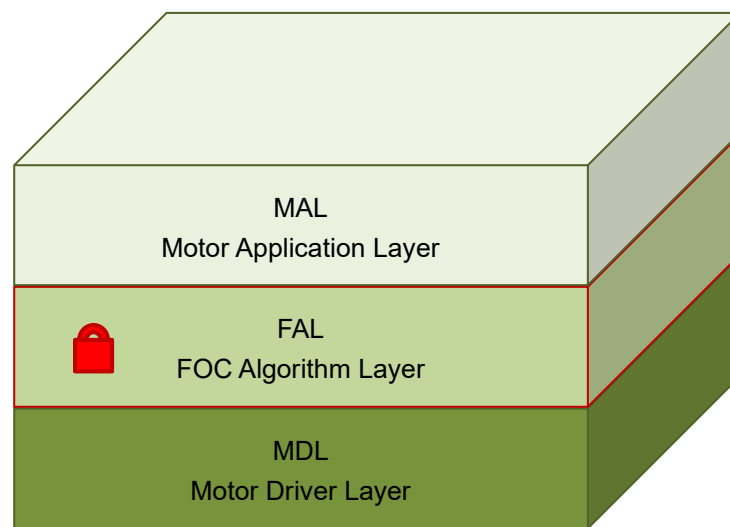


Figure 1.2 Layered architecture of ABOV library

Since the FOC library layers are not available at the code level, a software application is required for internal parameter configuration.

The ABOV 1-Shunt FOC Library supports aFDT (ABOV FOC Development Tool) to configure the internal parameters of the library. aFDT enables more intuitive and convenient configuration of internal parameters compared to modification at the code level.

aFDT will support real-time monitoring and tuning for easy motor tuning

1.3 Compatible MCUs

FOC Library is compatible with A34M41x, A33M11x, AC33Mx128, AC33Mx064 and AC31M1x64. FOC Library will be updated to support the entire motor series.

Table 1.2 ABOV M Series MCUs

Part No.	AC30M1x64	AC33Mx064	AC33Mx128	A33M11x	A34M41x
CPU Core	Cortex-M0	Cortex-M3	Cortex-M3	Cortex-M3	Cortex-M4F
Clock Speed	40MHz	48MHz	72MHz	96MHz	120MHz
Flash / SRAM	64KB / 4KB	64KB / 8KB	128KB / 12KB	256KB + 32KB / 16KB	512+32KB / 64KB
Timer	16-bit x 4 32-bit WDT 32-bit FRT	16-bit x 6 32-bit WDT	16-bit x 6 32-bit WDT	16-bit x 8 32-bit FRT 32-bit WDT	16-bit x 10 32-bit FRT x 2 32-bit WDT
DMAC	-	4-ch	15-ch	8-ch	16-ch
UART SPI I2C	2 / 1 / 1	2 / 1 / 1	4 / 2 / 2	4 / 2 / 2	6 / 3 / 2
3P Motor PWM	1	1	2	2	2
GPIO	44	44	64	56	107
Special Functions	64/32-bit H/W Divider SubOSC	-	PGA x 4 COMP x 4	QE1 x 2 OPAMP & COMP x 4	CAN 2.0B QE1 x 2 PGA x 3 COMP x 4
Others	IEC60730 CRC16	IEC60730 CRC16	IEC60730 CRC16	IEC60730 CRC16	IEC60730 CRC16 AES-128
ADC	12-bit 1Msps	12-bit 1.5Msps	12-bit 1.5Msps	12-bit 1.5Msps	12-bit 1.5Msps
Internal OSC	40MHz (±3%)	1MHz RingOSC	20MHz (±3%)	32MHz (±1.5%)	32MHz (±3%)
Analog IP	POR/VDC/LVD	POR/VDC/LVD	POR/VDC/LVD	POR/VDC/LVD	POR/VDC/LVD
Op. Temp.	-40 to +85/105°C	-40 to +85/105°C	-40 to +85/105°C	-40 to +105°C	-40 to +85°C
Op.Voltage	2.4 to 5.5V	3.0V to 5.5V	3.0V to 5.5V	2.5V to 5.5V	2.5V to 5.5V
PKG	LQFP-48,QFP-32 SOP28,SOP24	LQFP-48 LQFP-32	LQFP-80 LQFP-64	LQFP-64 LQFP-48	LQFP-120 LQFP-100 LQFP-64
Status	MP	MP	MP	MP	MP
Applications	Fan,Pump, Low cost FOC	Air Con., Refrigerator, White goods	Air Con. Compressor, WashingM/C FOC	Compressor, System Aircon High speed FOC	System Aircon High speed FOC IoT App.

1.4 Getting Started

1.4.1 KEIL

- 1) Copy the *.FLM file from the \Flashloader\KEIL folder to C:\Keil_v5\ARM\Flash.
- 2) Go to \Examples\Motor_Application\Keil, and open the desired MCU folder.

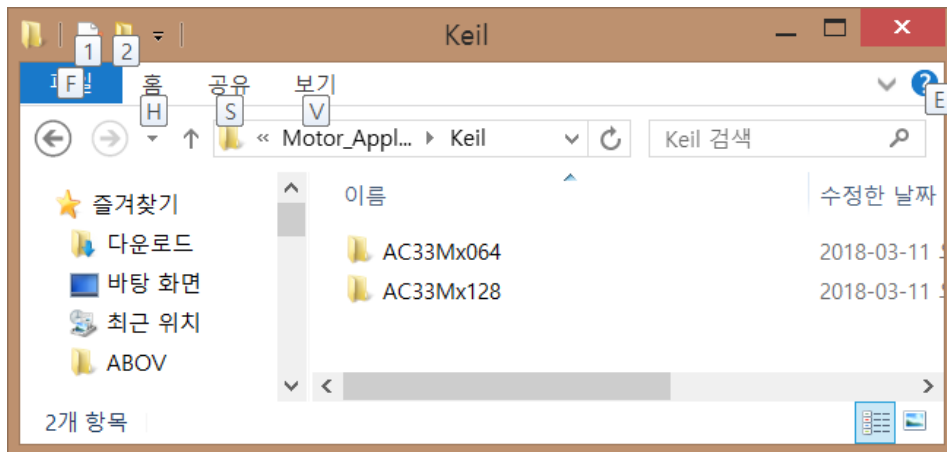


Figure 1.3 Project folder

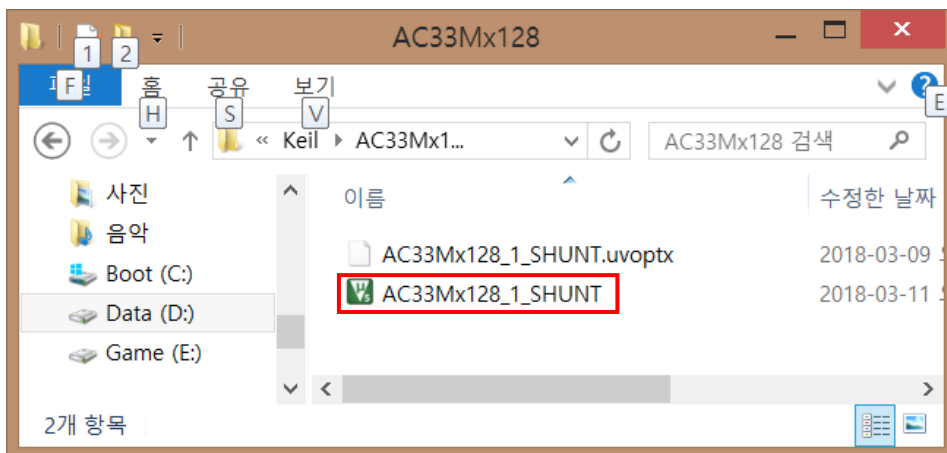


Figure 1.4 Project file

- 3) Open the project file, and set up the Debugger as shown below.

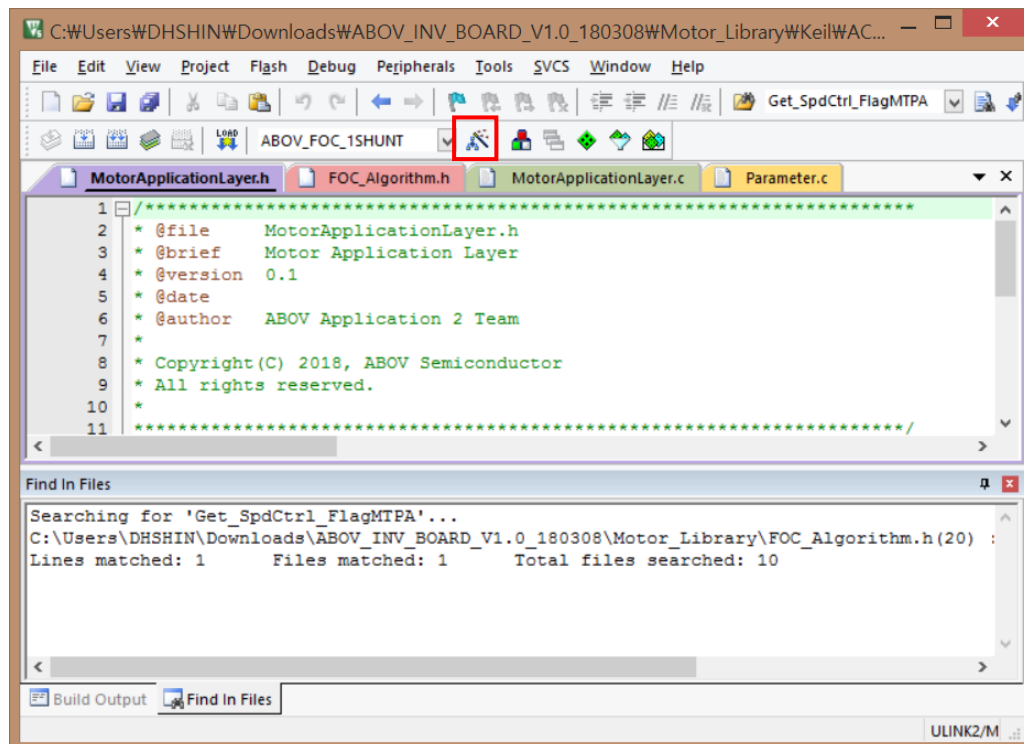


Figure 1.5 KEIL project options

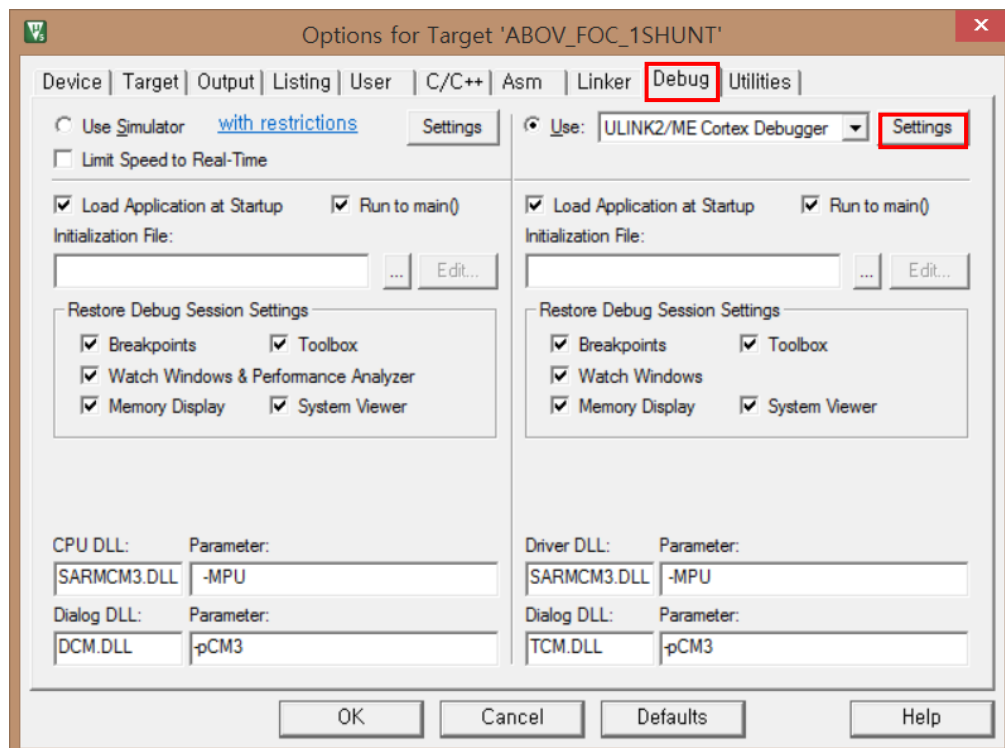


Figure 1.6 Project options – Debug tab

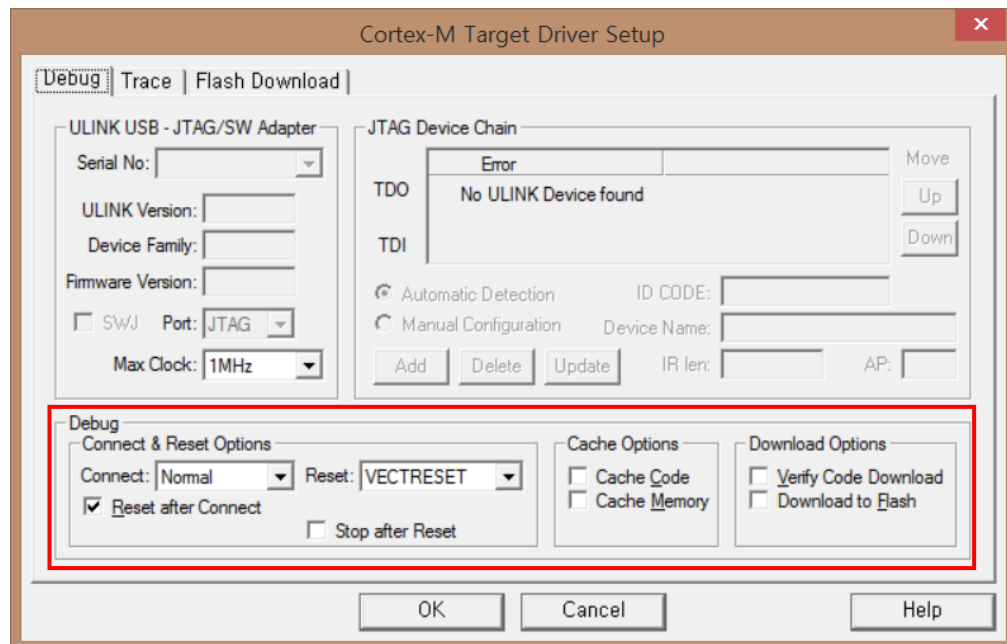


Figure 1.7 Debugger configuration

- 4) Set up the Flash loader as shown below.

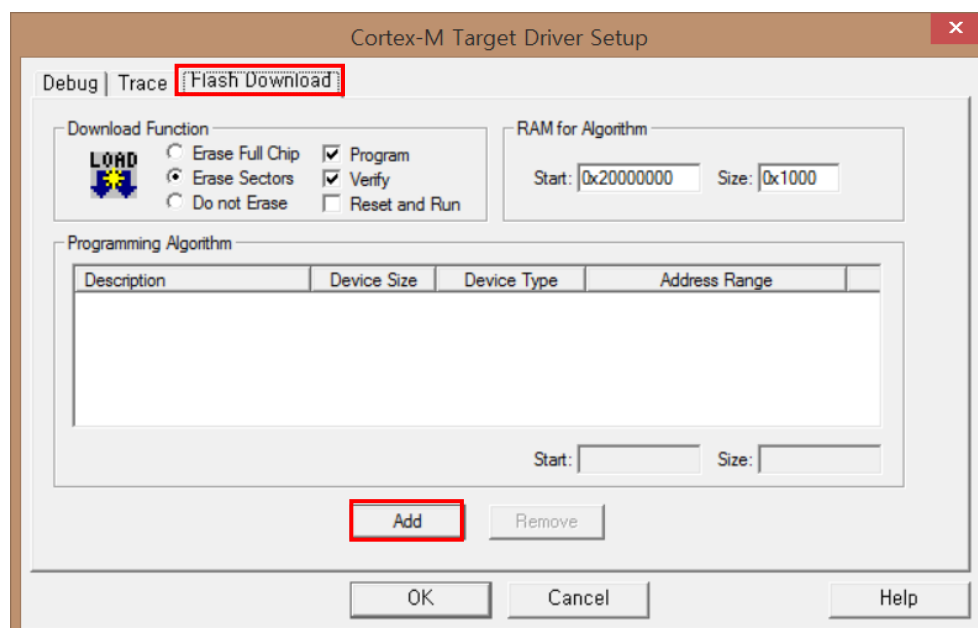


Figure 1.8 Debugger configuration – Flash Download tab

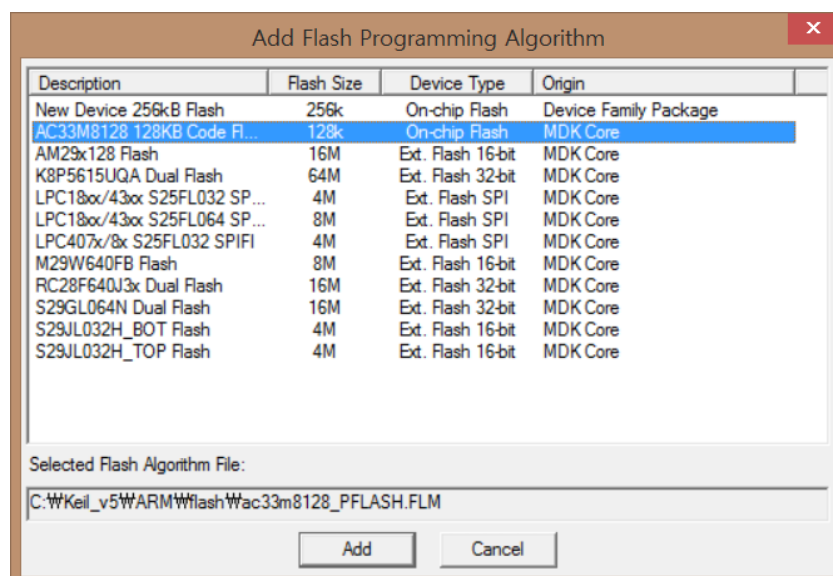


Figure 1.9 Configuration of ABOV MCU Flash loader

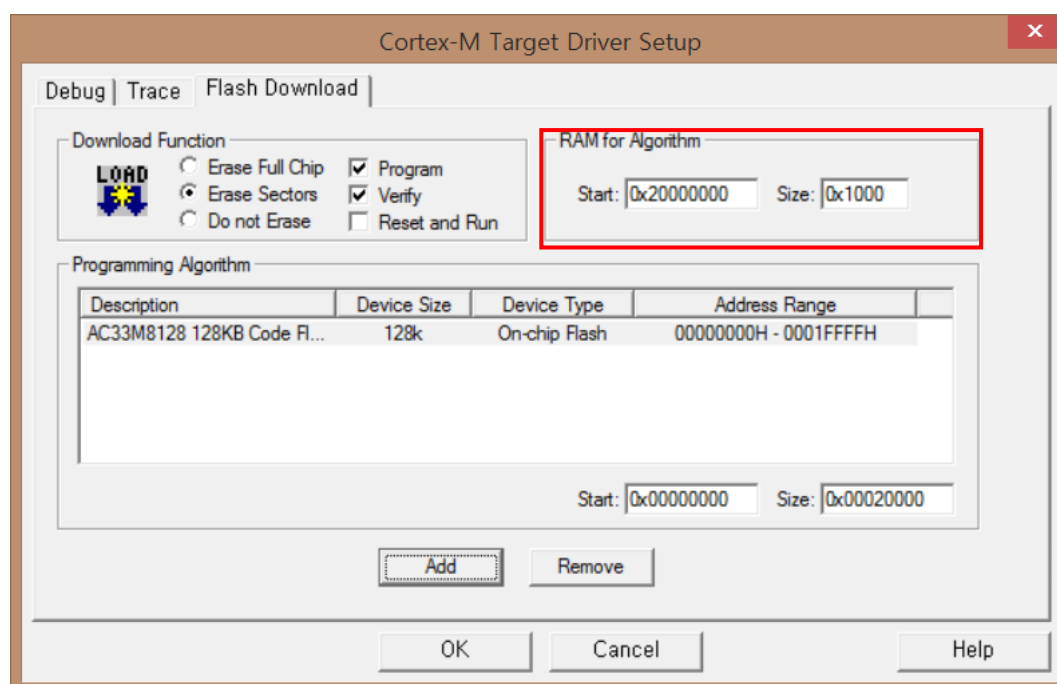


Figure 1.10 Configuration of Flash loader upload path

1.4.2 IAR

- 1) Copy the config folder in \Flashloader\IAR to C:\Program Files (x86)\IAR Systems\Embedded Workbench x.x\arm (IAR Workbench installation folder).
- 2) Go to \Examples\Motor_Application\IAR, and open your desired MCU folder.

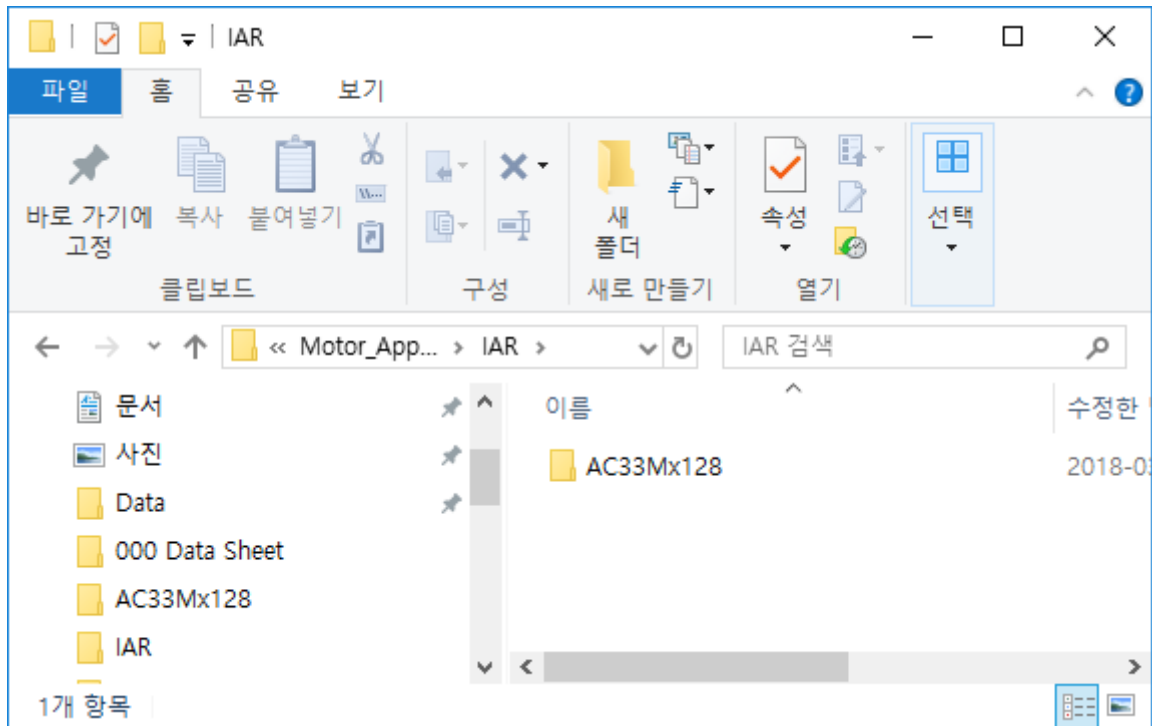


Figure 1.11 IAR project folder

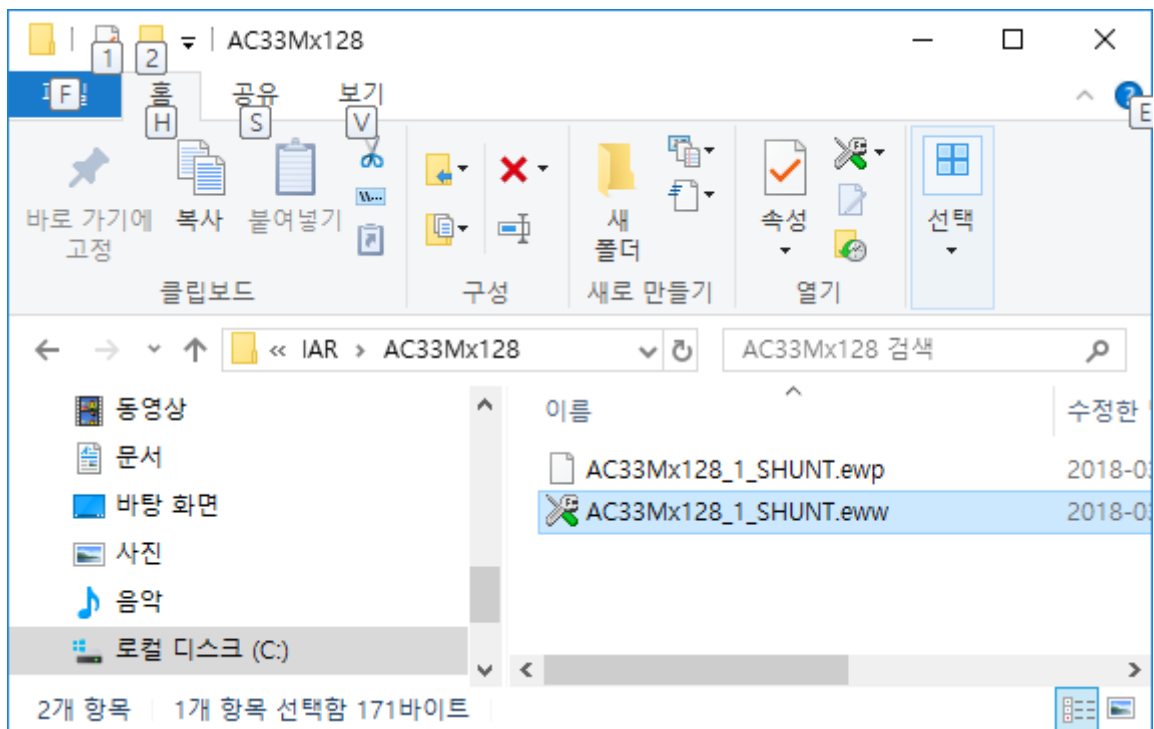


Figure 1.12 IAR project file

- 3) Open the project file, and set up the project options as shown below.

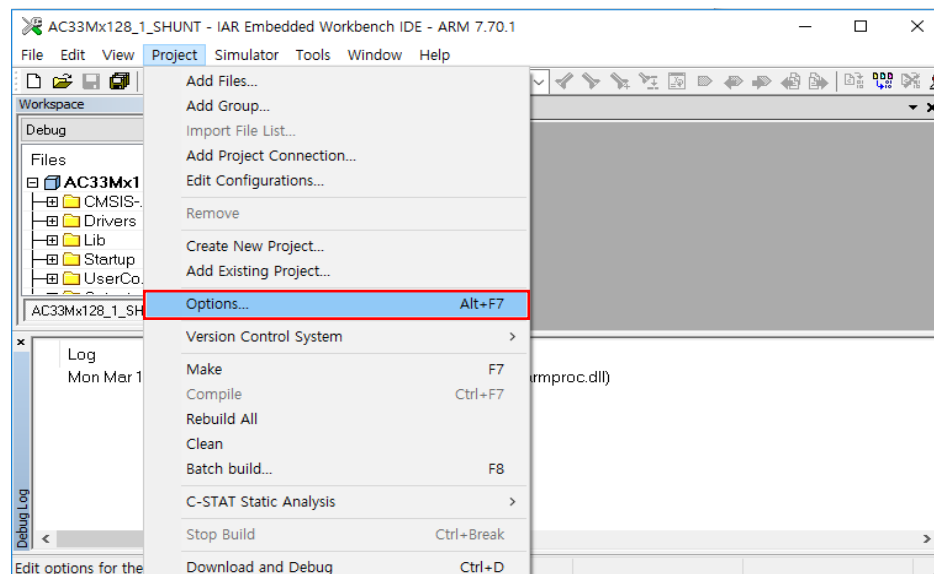


Figure 1.13 IAR project options

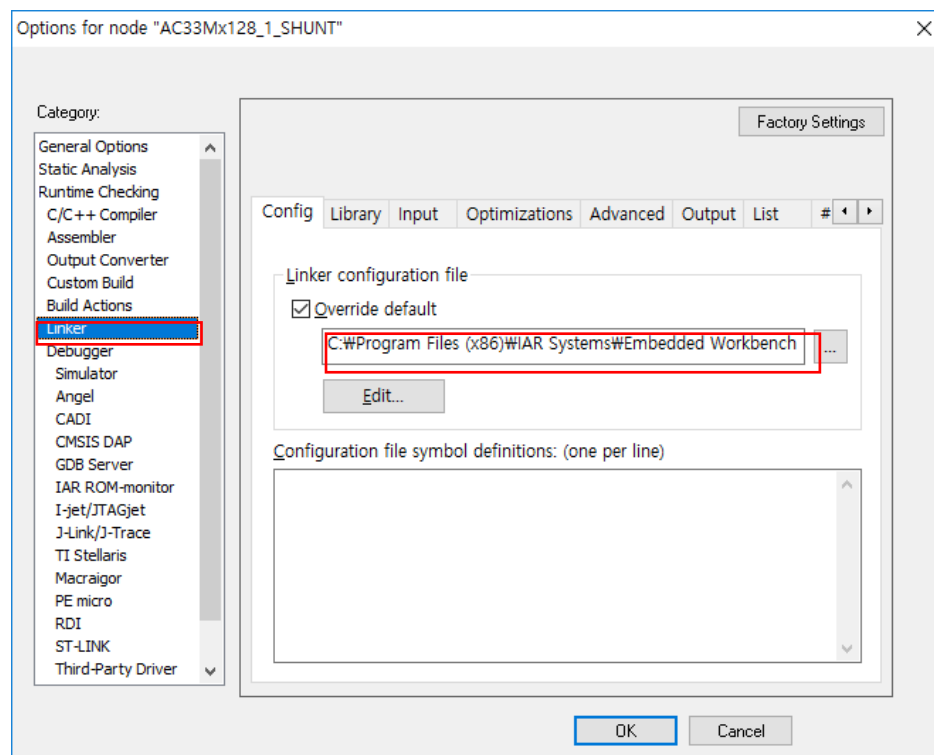


Figure 1.14 Designation of IAR linker configuration file (requires change of default installation path)

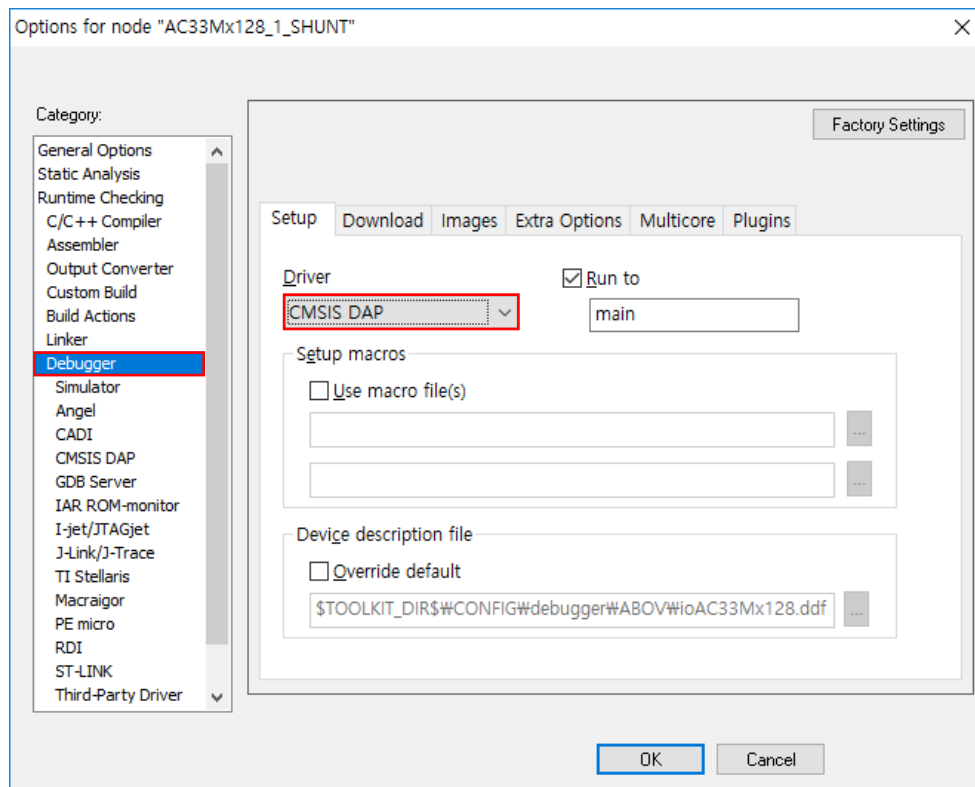


Figure 1.15 IAR project debugger configuration

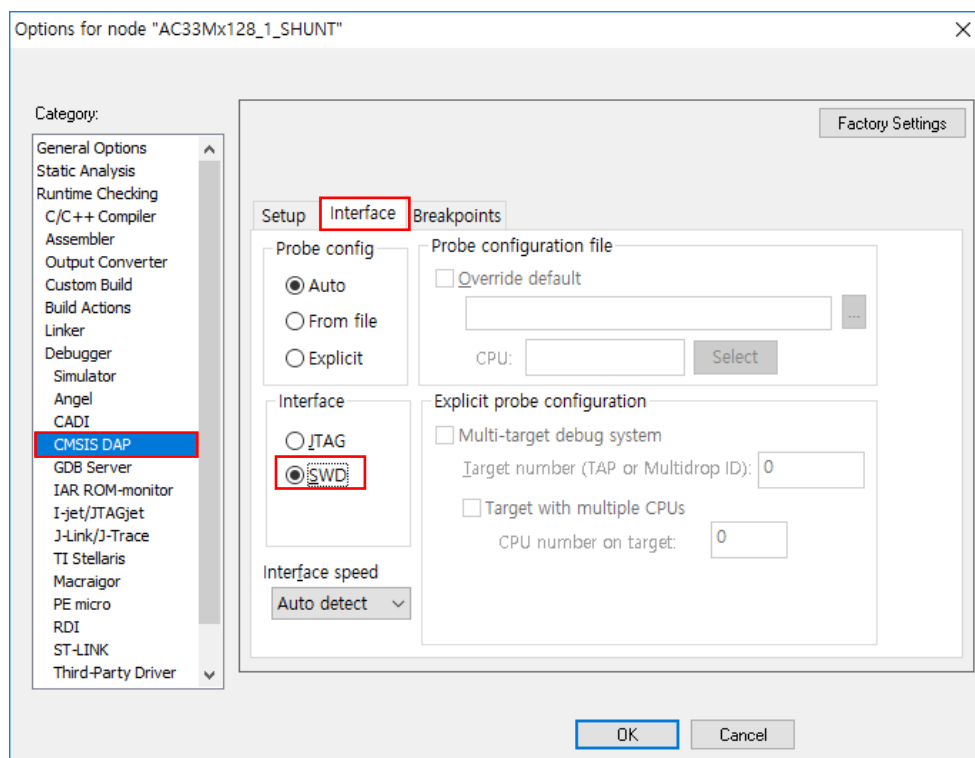


Figure 1.16 IAR project CMSIS-DAP configuration

Chapter 2. MDL (Motor Driver Layer)

2.1 Overview

The MDL (Motor Driver Layer) contains MCU-specific functions that initialize and control internal registers for motor driving. The MDL code specifies the MCU type used in project, and thus it must be pre-defined in project option as follows:

`__DEF_(product code)__`

For the AC33Mx128 series for instance, `__DEF_AC33Mx128__` is pre-defined in the project file, and the MDL code is placed between `#ifdef __DEF_AC33Mx128__` and `#endif`.

When users open the desired project from among the provided project files, the MDL of the product is automatically applied during compilation as it is pre-defined under project options.

The MDL consists mainly of codes for port configuration, MPWM block control, and ADC block control. It also features codes for the configuration of other registers.

NOTE: This layer must be modified according to the MCU pin mapping if the motor control board is changed.

2.2 Description of Functions

This section describes the basic functions of the MDL layer. The examples are based on AC33Mx064. For more details, refer to the source codes of ABOV 1-Shunt FOC Library.

2.2.1 MPWMn_IRQHandler

Name of Function	MPWMn_IRQHandler (n = 0,1,2...)	
Argument	void	
Return Value	void	
Description	This function configures MPWM interrupt handlers used in the FOC library. For configuration, define the MPWM interrupt handler, and call MAL_MPWM_Handler via the defined handler.	
Code Example	<pre>// For the case of MPWM0 void MPWM0_IRQHandler(void) { #ifdef __DEF_AC33Mx064__ MAL_MPWM_Handler(); #endif } // For the case of MPWM1 void MPWM1_IRQHandler(void) { #ifdef __DEF_AC33Mx064__ MAL_MPWM_Handler(); #endif }</pre>	

2.2.2 MDL_getMPWMStatus

Name of Function	MDL_getMPWMStatus	
Argument	void	
Return Value	uint16_t	MPWM status register value
Description	This function returns the MPWM status register value used in the FOC library.	
Code Example	<pre>// For the case of MPWM0 uint16_t MDL_getMPWMStatus(void) { return MP0->SR; } // For the case of MPWM1 uint16_t MDL_getMPWMStatus(void) { #ifdef __DEF_AC33Mx064__ return MP1->SR; #endif }</pre>	
Application Example	<pre>uint16_t status; status = MDL_getMPWMStatus();</pre>	

2.2.3 MDL_clearMPWMStatus

Name of Function	MDL_getMPWMStatus	
Argument	uint16_t status	MPWM status register clear value
Return Value	void	
Description	This function clears the MPWM status register used in the FOC library.	
Code Example	<pre>// For the case of MPWM0 void MDL_clearMPWMStatus(uint16_t status){ #ifdef __DEF_AC33Mx064__ MP0->SR = status; #endif } // For the case of MPWM1 void MDL_clearMPWMStatus(uint16_t status){ #ifdef __DEF_AC33Mx064__ MP1->SR = status; #endif }</pre>	
Application Example	MDL_clearMPWMStatus (1<<6); // Clear bit No. 6	

2.2.4 MDL_getADCShunt

Name of Function	MDL_getADCShunt	
Argument	uint16_t num	Requests the resulting value from the numth shunt resistor channel ADC
Return Value	uint16_t	The resulting value from the numth shunt resistor channel ADC
Description	This function returns a resulting value from shunt resistor channel ADC used in the FOC library. The numth shunt resistor channel ADC by the MPWM trigger is returned.	
Code Example	<pre>uint16_t MDL_getADCShunt(uint16_t num) { #ifdef __DEF_AC33Mx064__ if(num == 2) return (AD0->DR0>>4); else if(num == 1) return (AD0->DR1>>4); #endif }</pre>	
Application Example	<pre>uint16_t shunt1_tmp, shunt2_tmp; shunt1_tmp = MDL_getMPWMStatus(1); shunt2_tmp = MDL_getMPWMStatus(2);</pre>	

2.2.5 MDL_checkFault

Name of Function	MDL_checkFault	
Argument	void	
Return Value	uint16_t	Status value of external fault signal
Description	This function returns the value of the GPIO port connected to an external fault signal. Since 0 is usually a fault, the software application used to call this function recognizes 0 as a fault.	
Code Example	<pre>uint32_t MDL_checkFault(void) { #ifdef __DEF_AC33Mx064__ return ((PB->IDR >> 6)& 0x01); // Where PB6 is used as the fault signal #endif }</pre>	
Application Example	<pre>if (MDL_checkFault() == 0){ // Where the return value is 0 // Fault signal processing }</pre>	

2.2.6 MDL_pwmON

Name of Function	MDL_pwmON	
Argument	void	
Return Value	void	
Description	This function activates the MPWM output used in the FOC library.	
Code Example	<pre>void MDL_pwmON(void) { #ifdef __DEF_AC33Mx064__ MP0->PSR0 = 0xCA00; #endif }</pre>	
Application Example	MDL_pwmON() // Activate MPWM output	

2.2.7 MDL_pwmOFF

Name of Function	MDL_pwmOFF	
Argument	void	
Return Value	void	
Description	This function deactivates the MPWM output used in the FOC library.	
Code Example	<pre>void MDL_pwmOFF(void) { #ifdef __DEF_AC33Mx064__ MP0->PSR0 = 0xCA3F; #endif }</pre>	
Application Example	MDL_pwmOFF() // Deactivate MPWM output	

2.2.8 MDL_ATRInit

Name of Function	MDL_ATRInit	
Argument	void	
Return Value	void	
Description	This function initializes MPWM trigger values used in the FOC library.	
Code Example	<pre>void MDL_ATRInit() { #ifdef __DEF_AC33Mx064__ MP0->ATR1 = (1<<16) ((10) & 0xFFFF); MP0->ATR2 = (1<<16) (((MP0->PRD)-10) & 0xFFFF); #endif }</pre>	
Application Example	MDL_ATRInit() // Initialize MPWM trigger	

2.2.9 MDL_setATR

Name of Function	MDL_setATR	
Argument	void	
Return Value	void	
Description	This function sets the trigger value calculated in the FOC algorithm layer as the MPWM trigger value for the FOC library.	
Code Example	<pre>void MDL_setATR() { #ifdef __DEF_AC33Mx064__ MP0->ATR1 = (1<<16) ((Get_Tsamp_NUM(1)) & 0xFFFF); MP0->ATR2 = (1<<16) ((Get_Tsamp_NUM(2)) & 0xFFFF); #endif }</pre>	
Application Example	MDL_setATR()	

2.2.10 MDL_AllHalfPeriod

Name of Function	MDL_AllHalfPeriod	
Argument	void	
Return Value	void	
Description	This function sets all duty values of the MPWM used in the FOC library to half of the PWM period. It is executed during initialization to prevent the motor from running.	
Code Example	<pre>void MDL_AllHalfPeriod(void) { #ifdef __DEF_AC33Mx064__ MP0->DUH = PWM_HALF_PRD; MP0->DUL = PWM_HALF_PRD; MP0->DVH = PWM_HALF_PRD; MP0->DVL = PWM_HALF_PRD; MP0->DWH = PWM_HALF_PRD; MP0->DWL = PWM_HALF_PRD; #endif }</pre>	
Application Example	MDL_AllHalfPeriod()	

2.2.11 MDL_writePWMDuty

Name of Function	MDL_writePWMDuty	
Argument	uint16_t *duty_array	Six array elements to be set as the MPWM duty values
Return Value	void	
Description	This function sets the duty value calculated in the FOC algorithm layer as the MPWM duty value used in the FOC library.	
Code Example	<pre>void MDL_writePWMDuty(uint16_t *duty_array) { #ifdef __DEF_AC33Mx064__ MP0->DWH = duty_array[0]; MP0->DUH = duty_array[2]; MP0->DVH = duty_array[1]; MP0->DWL = duty_array[3]; MP0->DVL = duty_array[4]; MP0->DUL = duty_array[5]; #endif }</pre>	
Application Example	<pre>uint16_t mDuty[6]; Get_Duty(mDuty); // Final duty value obtained from the FOC Algorithm Layer MDL_writePWMDuty(mDuty); // Writes to the MPWM duty register</pre>	

2.2.12 MDL_getADC

Name of Function	MDL_getADC	
Argument	uint8_t ad_ch	ADC channel
Return Value	uint16_t	Final value of 12-bit ADC
Description	This function returns the value resulting from ADC through ad_ch. It is used to read the VDC_LINK value.	
Code Example	<pre>uint16_t MDL_getADC(uint8_t ad_ch) { #ifdef __DEF_AC33Mx064__ AD1->SCSR = ad_ch; // select ch ADC_StartCmd(AD1); while((AD1->SR&0x40)==0); //wait until End-of-Conversion while((AD1->SR&0x80)==0); //wait until End-of Conversion return (ADC_GetData(AD1)>>4); // using 12bit #endif }</pre>	
Application Example	<pre>#define AD_VDC_LINK (7) // ADC channel for the VDC_LINK value int16_t AD_Vdc; AD_Vdc = MDL_getADC(AD_VDC_LINK);</pre>	

2.2.13 MDL_DMA_reset

Name of Function	MDL_DMA_reset	
Argument	void	
Return Value	void	
Description	This function conducts DMA reset, which is necessary for the MPWM trigger to get ADC values read. It is required only for AC33Mx128, and does not perform any action in other MCUs. Using the function without modification is recommended.	
Application Example	MDL_DMA_reset();	

2.2.14 MDL_ADC_init

Name of Function	MDL_ADC_init	
Argument	void	
Return Value	void	
Description	This function initializes the ADC used in the FOC library. If the board is changed, code modification according to pin mapping is required.	
Application Example	MDL_ADC_init();	

2.2.15 MDL_MPWM_init

Name of Function	MDL_MPWM_init	
Argument	void	
Return Value	void	
Description	This function initializes the MPWM used in the FOC library. If the board is changed, code modification according to pin mapping is required.	
Application Example	MDL_MPWM_init();	

2.2.16 MDL_PCU_Init

Name of Function	MDL_PCU_Init	
Argument	void	
Return Value	void	
Description	This function initializes each port. If the board is changed, code modification according to pin mapping is required.	
Application Example	MDL_PCU_Init();	

Chapter 3. FAL (FOC Algorithm Layer)

3.1 Overview

The FAL (FOC Algorithm Layer) is the FOC core algorithm layer.

This layer contains various functions essential for 1-Shunt FOC operations. Users can retrieve and use desired functions, but the code remains undisclosed.

Generally, the internal variables of FAL cannot be monitored. However, since the key variables are stored in MDL and FAL functions simply reference their addresses, the input/output variables of FAL functions can be monitored by declaring those variables in MDL, and they may be utilized in individual blocks.

And the Get function retrieves internal variables of FAL that need to be monitored, and the Set function configures those internal variables.

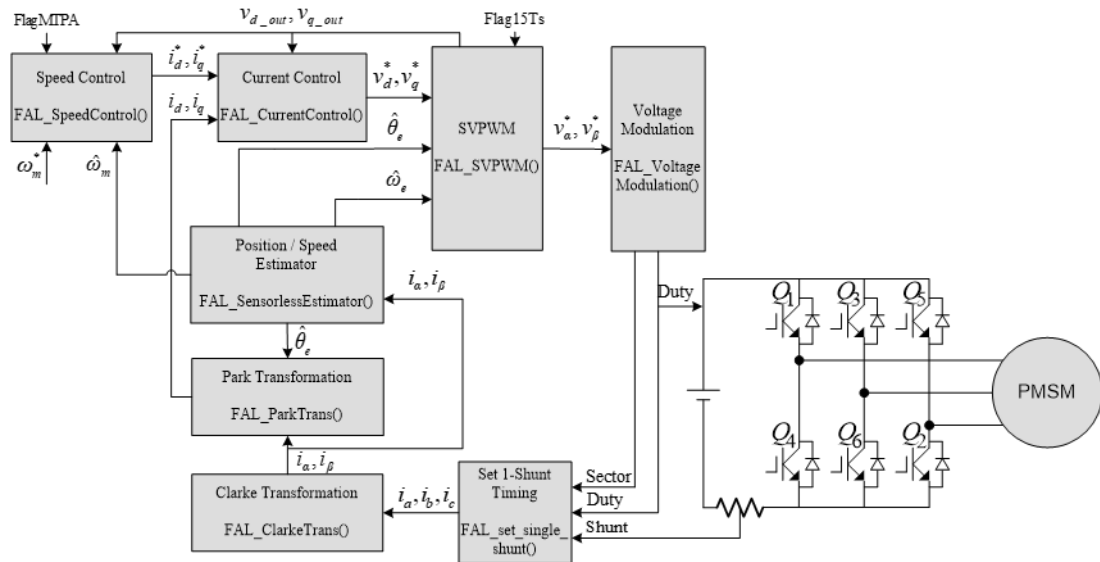


Figure 3.1 Block diagram of the 1-Shunt FOC function

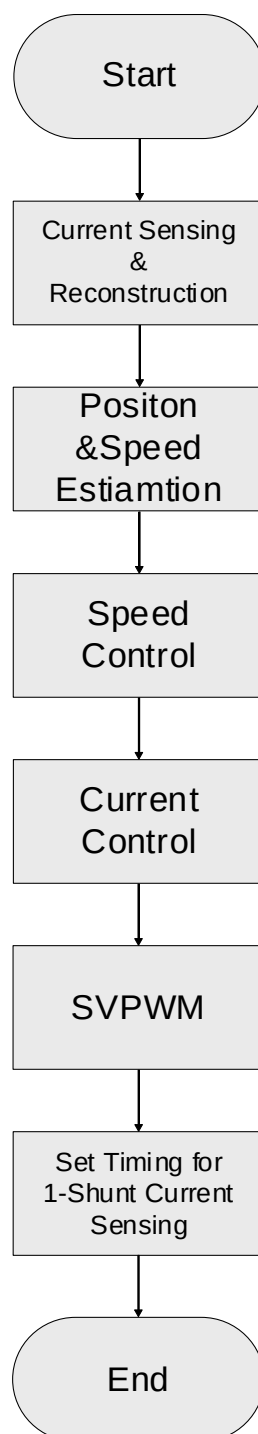


Figure 3.2 Flow chart of the 1-Shunt FOC algorithm

3.2 Description of Functions

This section describes the functions of FAL by functionality.

3.2.1 Main Functions of FAL

These functions essential for motor operations.

3.2.1.1 FAL_SensorlessEstimator

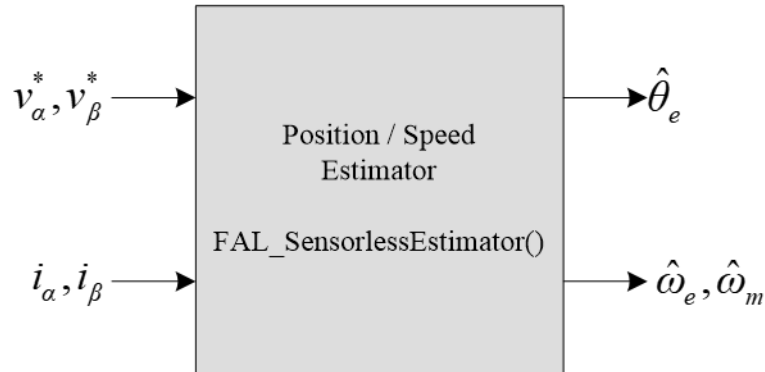


Figure 3.3 Block diagram of the Sensorless Estimator function

Name of Function	FAL_SensorlessEstimaor	
Argument	int16_t * Vdss_cmd_Q14	α -axis (stationary reference frame) commanded voltage
	int16_t * Vqss_cmd_Q14	β -axis (stationary reference frame) commanded voltage
	int16_t Idss_Q14	α -axis (stationary reference frame) current
	int16_t Iqss_Q14	β -axis (stationary reference frame) current
	int16_t * We_Q14	Present estimated electrical angular velocity $\hat{\omega}_e$ [rad/s]
	int16_t * Spd_actual_rpm	Present estimated mechanical angular velocity $\hat{\omega}_m$ [RPM]
	int16_t * Est_Theta_Q14	Present estimated angle $\hat{\theta}_e$ [radian]
Return Value	void	
Description	Estimates electrical angle $[\theta_e]$, electrical angular velocity $[\omega_e]$ and mechanical angular velocity $[\omega_m]$.	
Application Example	FAL_SensorlessEstimator(&Vdss_cmd_Q14, &Vqss_cmd_Q14, Idss_Q14, Iqss_Q14, &We_Q14, &Spd_actual_rpm, &Est_Theta_Q14);	

3.2.1.2 FAL_SpeedControl

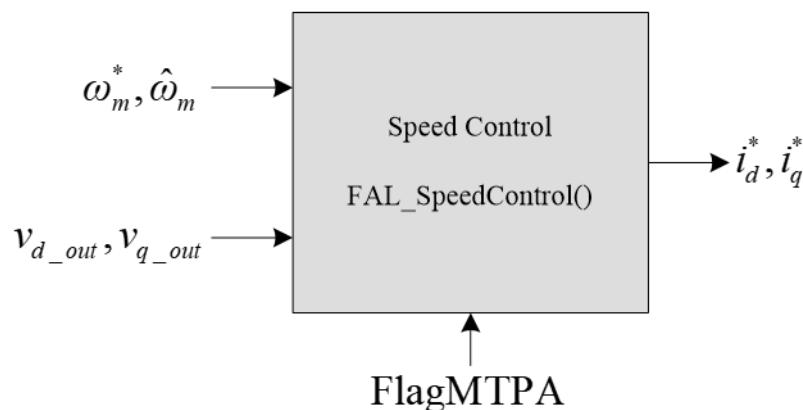


Figure 3.4 Block diagram of the Speed Control function

Name of Function	FAL_SpeedControl	
Argument	int16_t Speed_cmd_rpm	Commanded speed ω_m^* [RPM]
	int16_t Spd_actual_rpm	Present estimated speed $\hat{\omega}_m$ [RPM]
	int16_t Vdse_out_Q14	d -axis (synchronous reference frame) output voltage of the FAL_SVPWM function
	int16_t Vqse_out_Q14	q -axis (synchronous reference frame) output voltage of the FAL_SVPWM function
	int16_t * ldse_cmd	d -axis (synchronous reference frame) commanded current
	int16_t * lqse_cmd	q -axis (synchronous reference frame) commanded current
Return Value	void	
Description	Controls present speed [$\hat{\omega}_m$] to meet the commanded speed [ω_m^*]. AFDT can be used to configure the gain of the PI controller, maximum and minimum values of the q-axis commanded current, and MTPA Enable Flag.	
Application Example	FAL_SpeedControl(Spd_cmd_rpm, Spd_actual_rpm, Vdse_out_Q14, Vqse_out_Q14, &ldse_cmd_Q14, &lqse_cmd_Q14);	

3.2.1.3 FAL_CurrentControl

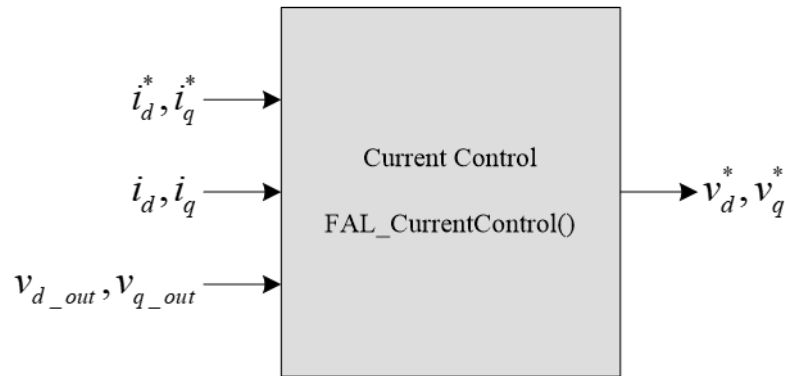


Figure 3.5 Block diagram of the Current Control function

Name of Function	FAL_CurrentControl	
Argument	int16_t Idse_cmd_Q14	d -axis (synchronous reference frame) commanded current
	int16_t Iqse_cmd_Q14	q -axis (synchronous reference frame) commanded current
	int16_t Idse_Q14	d -axis (synchronous reference frame) current
	int16_t Iqse_Q14	q -axis (synchronous reference frame) current
	int16_t Vdse_out_Q14	d -axis (synchronous reference frame) output voltage of the FAL_SVPWM function
	int16_t Vqse_out_Q14	q -axis (synchronous reference frame) output voltage of the FAL_SVPWM function
	int16_t* Vdse_cmd_Q14	d -axis (synchronous reference frame) commanded voltage
	int16_t* Vqse_cmd_Q14	q -axis (synchronous reference frame) commanded voltage
Return Value	void	
Description	Controls the current to meet the d, q -axis commanded current, which is the output of the speed controller. aFDT can be used to set the bandwidth of the current controller.	
Application Example	FAL_CurrentControl(Idse_cmd_Q14, Iqse_cmd_Q14, Idse_Q14, Iqse_Q14, Vdse_out_Q14, Vqse_out_Q14, &Vdse_cmd_Q14, &Vqse_cmd_Q14);	

3.2.1.4 FAL_SVPWM

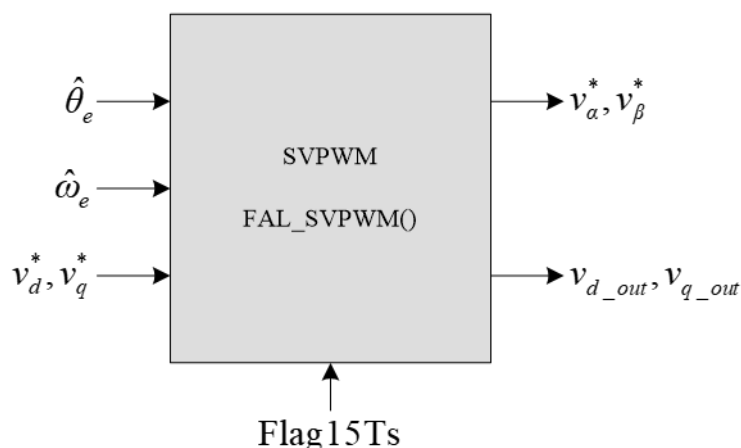


Figure 3.6 Block diagram of the SVPWM function

Name of Function	FAL_SVPWM	
Argument	int16_t Theta_Q14	Present electrical angle θ_e [radian]
	int16_t We_Q14	Estimated electrical angular velocity ω_e [rad/s]
	int16_t Vdse_cmd_Q14	d -axis (synchronous reference frame) commanded voltage
	int16_t Vqse_cmd_Q14	q -axis (synchronous reference frame) commanded voltage
	int16_t * Vdss_cmd_Q14	α -axis (stationary reference frame) commanded voltage
	int16_t * Vqss_cmd_Q14	β -axis (stationary reference frame) commanded voltage
	int16_t * Vdse_out_Q14	d -axis (synchronous reference frame) output voltage
	int16_t * Vqse_out_Q14	q -axis (synchronous reference frame) output voltage
Return Value	void	
Description	Performs space vector PWM. Conducts 1.5 Ts delay compensation at a speed desired by the user using the FAL_Set_Flag15Ts() function.	
Application Example	FAL_SVPWM(Theta_Q14, We_Q14, Vdse_cmd_Q14, Vqse_cmd_Q14, &Vdss_cmd_Q14, &Vqss_cmd_Q14, &Vdse_out_Q14, &Vqse_out_Q14);	

3.2.1.5 FAL_VoltageModulation

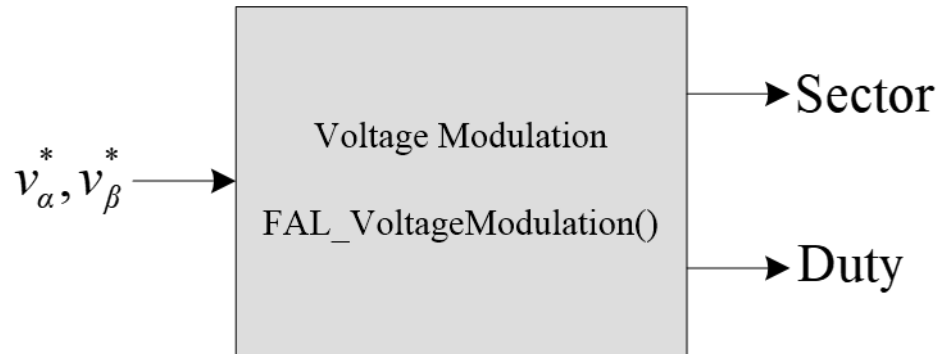


Figure 3.7 Block diagram of the Voltage Modulation function

Name of Function	FAL_VoltageModulationI	
Argument	int16_t * Vdss_cmd_Q14	α -axis (stationary reference frame) commanded voltage
	int16_t * Vqss_cmd_Q14	β -axis (stationary reference frame) commanded voltage
	uint8_t * Sector	Sector (No. 0–5) containing SVPWM's valid voltage vector
	uint16_t * Duty	Duty
Return Value	void	
Description	Identifies sectors, and modulates and compensates the commanded voltage to output duty.	
Application Example	FAL_VoltageModulation(&Vdss_cmd_Q14, &Vqss_cmd_Q14, &Sector, &Duty);	

3.2.1.6 FAL_Set_InitialPosition

Name of Function	FAL_Set_InitialPosition	
Argument	int16_t * Theta_Q14	Electrical angle θ_e [radian]
Return Value	void	
Description	Initializes the electrical angle θ_e to 0.	
Application Example	FAL_Set_InitialPosition(&Est_Theta_Q14);	

3.2.1.7 FAL_Align

Name of Function	FAL_Align	
Argument	int16_t * ldse_cmd_Q14	<i>d</i> -axis (synchronous reference frame) commanded current
	int16_t * lqse_cmd_Q14	<i>q</i> -axis (synchronous reference frame) commanded current
Return Value	void	
Description	Performs forced alignment according to values set by aFDT.	
Application Example	FAL_Align(&ldse_cmd_Q14, &lqse_cmd_Q14);	

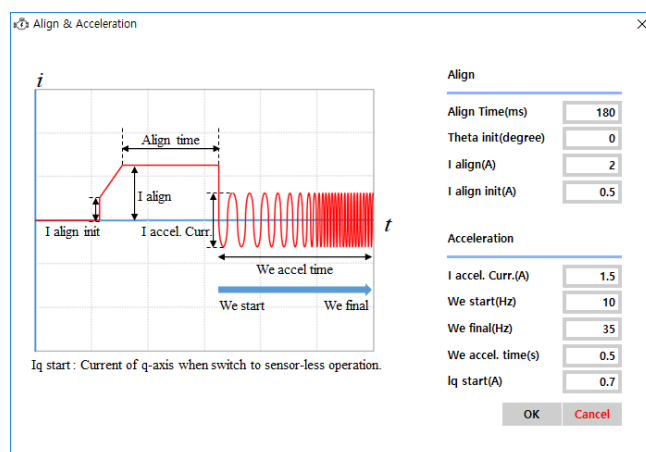


Figure 3.8 Align configuration of aFDT

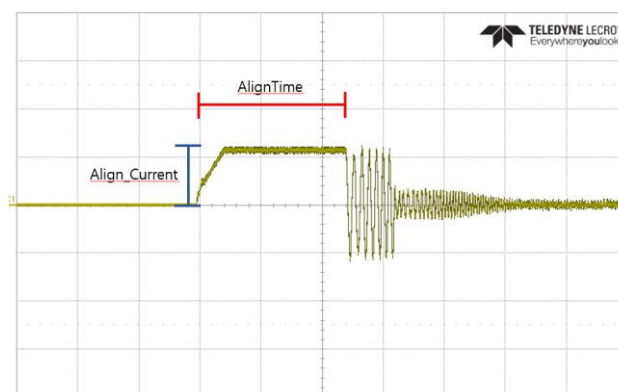


Figure 3.9 U-phase current waveform

3.2.1.8 FAL_Accelerate

Name of Function	FAL_Accelerate	
Argument	int16_t * Idse_cmd_Q14	<i>d</i> -axis (synchronous reference frame) commanded current
	int16_t * Iqse_cmd_Q14	<i>q</i> -axis (synchronous reference frame) commanded current
	int16_t * Acc_Theta_Q14	Synchronous acceleration angle θ_{acc}
Return Value	void	
Description	Performs synchronous acceleration according to values set by aFDT.	
Application Example	FAL_Accelerate(&Idse_cmd_Q14, &Iqse_cmd_Q14, &Acc_Theta_Q14);	

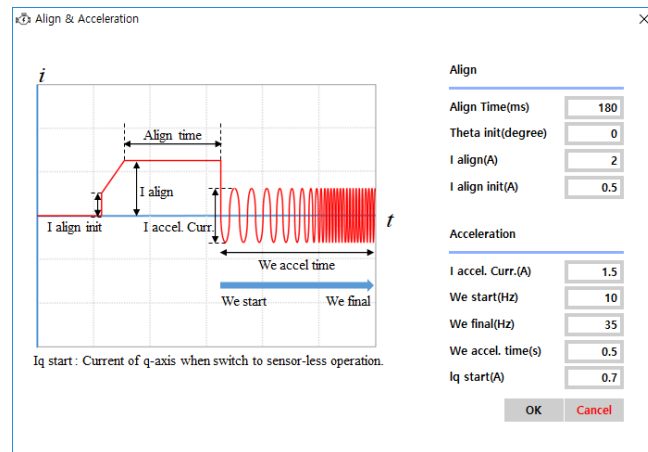


Figure 3.10 Acceleration configuration of aFDT

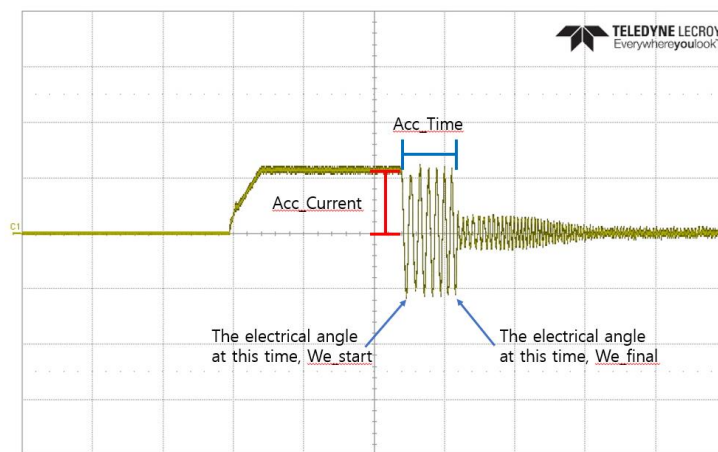


Figure 3.11 U-phase current waveform

3.2.1.9 FAL_ClarkeTrans

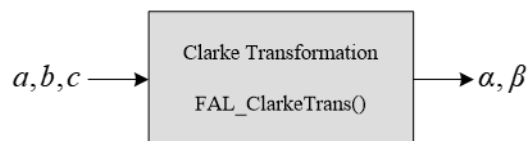


Figure 3.12 Block diagram of the Clarke Transformation function

Name of Function	FAL_ClarkeTrans	
Argument	int16_t A_Q14	A-axis
	int16_t B_Q14	B-axis
	int16_t C_Q14	C-axis
	int16_t * Alpah_Q14	α -axis
	int16_t * Beta_Q14	β -axis
Return Value	void	
Description	Converts 3-axis stationary reference frame coordinates into 2-axis stationary reference frame coordinates.	
Application Example	FAL_ClarkeTrans(las_Q14, lbs_Q14, lcs_Q14, &ldss_Q14, &lqss_Q14);	

3.2.1.10 FAL_InvClarkeTrans

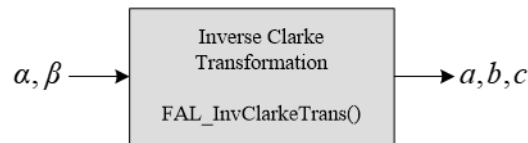


Figure 3.13 Block diagram of the Inverse Clarke Transformation function

Name of Function	FAL_InvClarkeTrans	
Argument	int16_t Alpha_Q14	α -axis
	int16_t Beta_Q14	β -axis
	int16_t * A_Q14	A-axis
	int16_t * B_Q14	B-axis
	int16_t * C_Q14	C-axis
Return Value	void	
Description	Converts 2-axis stationary reference frame coordinates into 3-axis stationary reference frame coordinates.	
Application Example	FAL_InvClarkeTrans(Idss_Q14, Iqss_Q14, &las_Q14, &lbs_Q14, &lcs_Q14);	

3.2.1.11 FAL_ParkTrans

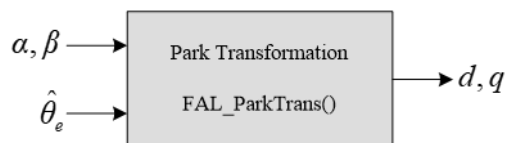


Figure 3.14 Block diagram of the Park Transformation function

Name of Function	FAL_ParkeTrans	
Argument	int16_t Alpha_Q14	α -axis
	int16_t Beta_Q14	β -axis
	int16_t Theta_Q14	Electrical angle θ_e
	int16_t * d_Q14	d -axis
	int16_t * q_Q14	q axis
Return Value	void	
Description	Converts 2-axis stationary reference frame coordinates to 2-axis synchronous reference frame coordinates.	
Application Example	FAL_ParkTrans(Idss_Q14, Iqss_Q14, Theta_Q14, &Idse_Q14, &Iqse_Q14);	

3.2.1.12 FAL_InvParkTrans

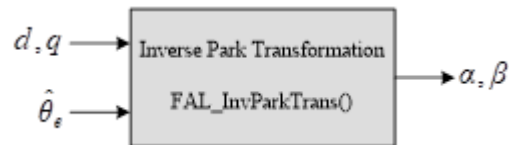


Figure 3.15 Block diagram of the Inverse Park Transformation function

Name of Function	FAL_InvParkeTrans	
Argument	int16_t d_Q14	d -axis
	int16_t q_Q14	q -axis
	int16_t Theta_Q14	Electrical angle θ_e
	int16_t* Alpha_Q14	α -axis
	int16_t* Beta_Q14	β -axis
Return Value	void	
Description	Converts 2-axis synchronous reference frame coordinates to 2-axis stationary reference frame coordinates.	
Application Example	FAL_InvParkTrans(Idse_Q14, Iqse_Q14, Theta_Q14, &Idss_Q14, &Iqss_Q14);	

3.2.1.13 FAL_Check_OverCurrent

Name of Function	FAL_Check_OverCurrent	
Argument	int16_t Ia	A-phase current calculated by the FAL_set_single_shunt function
	int16_t Ib	B-phase current calculated by the FAL_set_single_shunt function
	int16_t * Ias_Q14	A-phase current converted into Q14 form
	int16_t * Ibs_Q14	B-phase current converted into Q14 form
	int16_t * Ics_Q14	C-phase current converted into Q14 form
Return Value	int16_t	FaultFlag
Description	Converts 3-phase currents into the Q14 form, and returns whether they are an overcurrent. 0: Normal 1: Overcurrent	
Application Example	<pre> if (FAL_Check_OverCurrent(Ia, Ib, &Ias_Q14, &Ibs_Q14, &Ics_Q14)){ OC_cnt++; } else { OC_cnt = 0; } if (OC_cnt > 10) { // 100us * 10 = 1ms FAL_Set_op_mode(0); PWM_OFF(); } </pre>	

3.2.1.14 FAL_Init_variables

Name of Function	FAL_Init_variables	
Argument	uint16_t * Parameter	Array elements generated by aFDT
Return Value	void	
Description	Initializes various constants to values set by aFDT.	
Application Example	<pre> if(FAL_Get_op_mode == 0) FAL_Init_variables(Parameter) </pre>	

3.2.1.15 FAL_Current_Reconstruction

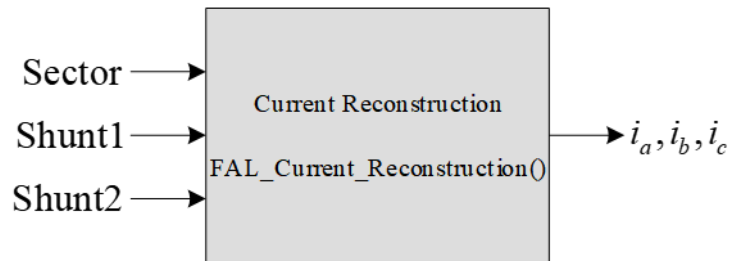


Figure 3.16 Block diagram of phase current calculation

Name of Function	FAL_Current_Reconstruction	
Argument	int16_t Sector	Sector (No. 0–5) containing SVPWM's valid voltage vector
	int16_t shunt1	Second ADC value of the current flowing through the shunt resistor
	int16_t shunt2	First ADC value of the current flowing through the shunt resistor
	int16_t * Ia	A-phase current value
	int16_t * Ib	B-phase current value
	int16_t * Ic	C-phase current value
Return Value	void	
Description	Calculates phase currents by reading the current flowing through the shunt resistor for the sector where the valid voltage vector lies.	
Application Example	FAL_Current_Reconstruction(Sector, shunt1, shunt2, (int16_t *)&Ia, (int16_t *)&Ib, (int16_t *)&Ic);	

3.2.1.16 FAL_Set_ADC_Trigger

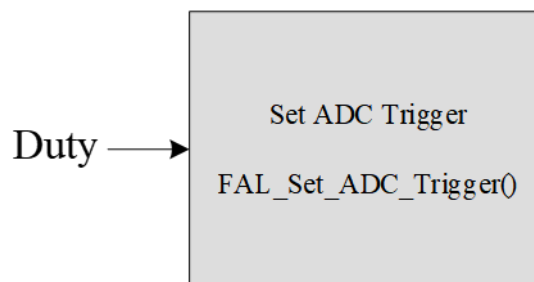


Figure 3.17 Block diagram of the Set ADC Trigger function

Name of Function	FAL_Current_Reconstruction	
Argument	int16_t* Duty	Duty
Return Value	void	
Description	For single-shunt current detection, two different ADC sampling points must be configured at every PWM cycle. ADC sampling points are calculated accordingly.	
Application Example	FAL_Set_ADC_Trigger((uint16_t *)Duty);	

3.2.2 Calculation Functions

Functions used to perform mathematical calculations.

3.2.2.1 FAL_Sine_Q15

Name of Function	FAL_Sine_Q15	
Argument	int16_t swTheta	Assumes the Q14 form, and falls in the range of -16384 to 16384. -16384 corresponds to $-\pi$, and 16384 to π .
Return Value	int16_t	Assumes the Q15 form, and falls in the range of -32768 to 32767. -32768 corresponds to -1, and 32767 to 1.
Description	Returns the sine value of the argument input in the Q15 form.	
Application Example	<pre>int16_t Theta; // Angle variable expressed in the Q14 form (Falls in the range of -16384 to 16384) int16_t Sin_Theta_Q15; // Return value of the Sine function expressed in the Q15 form Sin_Theta_Q15 = FAL_Sine_Q15(Theta);</pre>	

3.2.2.2 FAL_Cosine_Q15

Name of Function	FAL_Cosine_Q15	
Argument	int16_t swTheta	Assumes the Q14 form, and falls in the range of -16384 to 16384. -16384 corresponds to $-\pi$, and 16384 to π .
Return Value	int16_t	Assumes the Q15 form, and falls in the range of -32768 to 32767. -32768 corresponds to -1, and 32767 to 1.
Description	Returns the cosine value of the argument input in the Q15 form.	
Application Example	<pre>int16_t Theta; // Angle variable expressed in the Q14 form (Falls in the range of -16384 to 16384) int16_t Cos_Theta_Q15; // Return value of the Cosine function expressed in the Q15 form Cos_Theta_Q15 = FAL_Cosine_Q15(Theta);</pre>	

3.2.2.3 FAL_Cal_Vdc_Q14

Name of Function	FAL_Cal_Vdc_Q14	
Argument	int16_t Para	Resulting value from AD conversion of the DC-Link voltage
Return Value	int16_t	Assumes the Q14 form, and falls in the range of 0 to 16384. 16384 corresponds to VdcRef.
Description	Converts the AD conversion value of the DC-Link voltage to the Q14 form.	
Application Example	<pre>int16_t AD_Vdc; // DC-Link voltage value after AD conversion int16_t Vdc_Q14; // DC-Link voltage value converted to the Q14 form Vdc_Q14 = FAL_Cal_Vdc_Q14(AD_Vdc);</pre>	

3.2.3 Configuration Functions

These functions are used to configure the internal parameters of FAL.

3.2.3.1 FAL_Set_op_mode

Name of Function	FAL_Set_op_mode	
Argument	uint8_t Para	
Return Value	void	
Description	Sets the operation mode of FAL. 0: IDLE state, in which PWM output stays off. 1: Forced alignment mode, in which a certain amount of current is flown through to align the rotor. 2: Synchronous acceleration mode, in which a synchronous magnetic field is created to accelerate the rotor. 3: Sensorless control mode	
Application Example	FAL_Set_op_mode(1); //Forced alignment mode	

3.2.3.2 FAL_Set_Bw_Daxis_CurrentCtrl

Name of Function	FAL_Set_Bw_Daxis_CurrentCtrl	
Argument	uint16_t * Parameter	Array elements generated by aFDT
Return Value	void	
Description	Sets the bandwidth of the D-axis current controller to the value set under “Current Controller Bandwidth” of aFDT. Parameters necessary for bandwidth configuration are stored as array elements by aFDT. BandWidth is the current controller's bandwidth applied during initialization and BandWidth1 may be set as a current controller bandwidth to be used under user-specified conditions by calling the FAL_Set_Bw_Daxis_CurrentCtrl function.	
Application Example	<pre>if (Spd_cmd_rpm < 2500) { FAL_Set_Bw_Daxis_CurrentCtrl(Parameter); };</pre>	

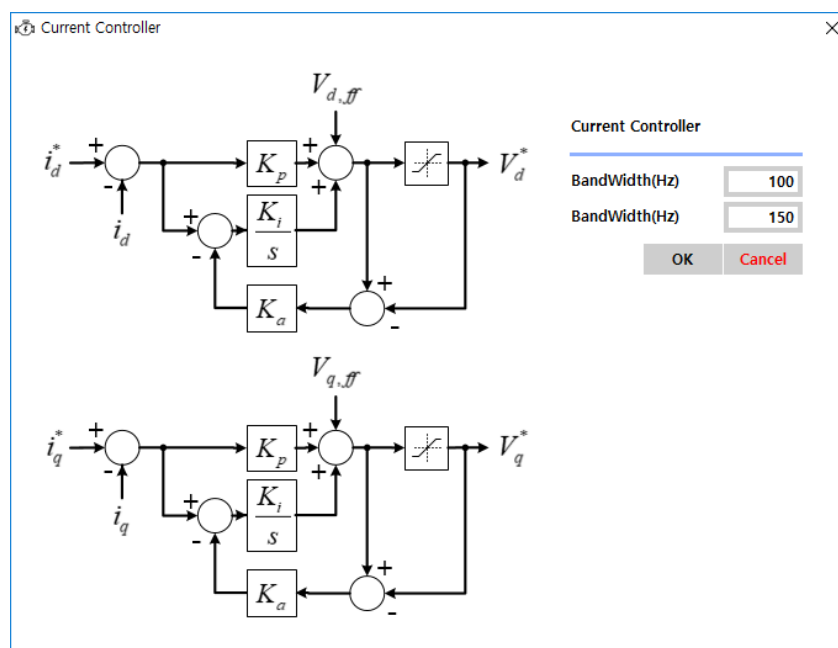


Figure 3.18 Configuration of current controller bandwidth of aFDT

3.2.3.3 FAL_Set_Bw_Qaxis_CurrentCtrl

Name of Function	FAL_Set_Bw_Daxis_CurrentCtrl	
Argument	uint16_t * Parameter	Array elements generated by aFDT
Return Value	void	
Description	Sets the bandwidth of the Q-axis current controller to the value set under "Current Controller Bandwidth" of aFDT. Parameters necessary for bandwidth configuration are stored as array elements by aFDT. BandWidth is the current controller's bandwidth applied during initialization and BandWidth1 may be set as a current controller bandwidth to be used under user-specified conditions by calling the FAL_Set_Bw_Qaxis_CurrentCtrl function.	
Application Example	<pre>if (Spd_cmd_rpm < 2500) { FAL_Set_Bw_Qaxis_CurrentCtrl(Parameter); };</pre>	

3.2.3.4 FAL_Set_Bw_EmfEstimator

Name of Function	FAL_Set_Bw_EmfEstimator	
Argument	uint16_t * Parameter	Array elements generated by aFDT
Return Value	void	
Description	Sets the estimator bandwidth to the value set under “Estimator” of aFDT. Parameters necessary for bandwidth configuration are stored as array elements by aFDT. Gr, Zeta, and Wn are applied to the current controller during initialization, and Gr1, Zeta1, and Wn1 may be set as current controller variables to be used under user-specified conditions by calling the FAL_Set_Bw_EmfEstimator function. LPF_Wspd is the bandwidth of the low pass filter applied to filter output values of the PI controller in the Theta PLL process.	
Application Example	<pre>if(FAL_Get_op_mode()>2) { FAL_Set_Bw_EmfEstimator(Parameter); }</pre>	

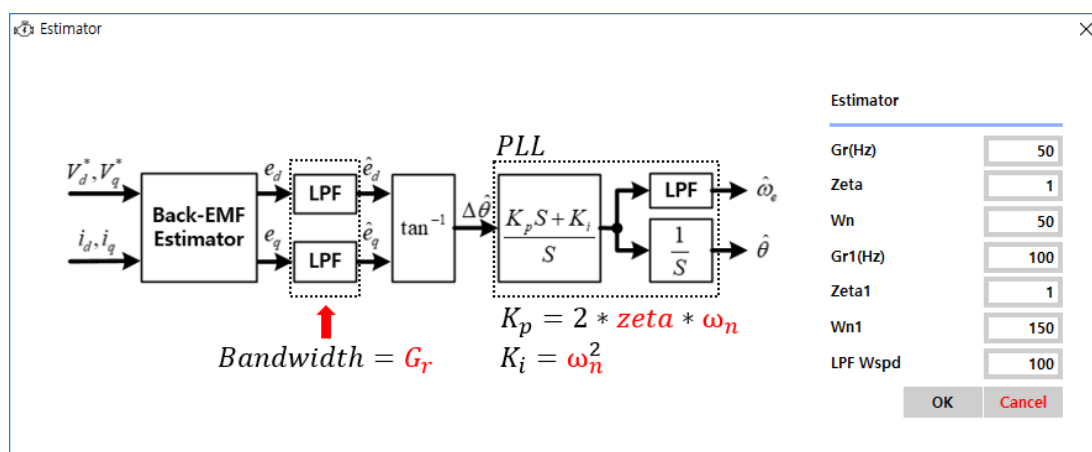


Figure 3.19 Configuration of Estimator of aFDT

3.2.3.5 FAL_Set_Bw_PLL

Name of Function	FAL_Set_Bw_PLL	
Argument	uint16_t * Parameter	Array elements generated by aFDT
Return Value	void	
Description	Sets the bandwidth of Phase Locked Loop of the estimator. Parameters used as an argument are stored as array elements by aFDT.	
Application Example	FAL_Set_Bw_PLL(Parameter);	

3.2.3.6 FAL_Set_Flag15Ts

Name of Function	FAL_Set_Flag15Ts	
Argument	uint8_t Para	0 or 1
Return Value	void	
Description	Sets the 1.5 Ts delay compensation flag of SVPWM. 0: Disabled 1: Enabled	
Application Example	<pre>if (Spd_cmd_rpm > 1500) { FAL_Set_Flag15TS(1); }</pre>	

3.2.3.7 FAL_Set_C15Ts_Qx

Name of Function	FAL_Set_C15Ts_Qx	
Argument	uint16_t	C15Ts_Qx
Return Value	void	
Description	Sets the constant to be used in time delay compensation for digital PWM as an internal variable of the algorithm. The time delay compensation constant is calculated by aFDT and stored as an array element, and can be retrieved using the FAL_Get_Target_C15Ts_Qx function. PWM time delay compensation is required for high speed motor operations, but a sudden introduction of PWM time delay compensation may cause a rapid change in the commanded voltage, thus producing pulsating currents. When starting to compensate for PWM time delay, therefore, pulsating currents may need to be reduced by retrieving the current control constant and gradually increasing it to reach the target value (use the FAL_Set_C15Ts_Qx function to apply the time delay compensation constant within the algorithm).	
Application Example	<pre> int16_t Spd_cmd_rpm //Commanded speed variable int16_t MAL_C15Ts_Qx; //Time delay compensation constant if (FAL_Get_Flag15Ts() == 0) { if (Spd_cmd_rpm >= 3000) { FAL_Set_Flag15Ts(1); } } else { if (Spd_cmd_rpm < 2500) { FAL_Set_Flag15Ts(0); } } if(FAL_Get_Flag15Ts()) { if (MAL_C15Ts_Qx < FAL_Get_Target_C15Ts_Qx()) { MAL_C15Ts_Qx++; } } else { if (MAL_C15Ts_Qx > 0) { MAL_C15Ts_Qx--; } } FAL_Set_C15Ts_Qx(MAL_C15Ts_Qx); //Time delay compensation constant applied //within algorithm using this function </pre>	

3.2.4 Read Functions

These functions are used to monitor internal parameters of FAL.

3.2.4.1 FAL_Get_op_mode

Name of Function	FAL_Get_op_mode	
Argument	void	
Return Value	uint8_t	op_mode
Description	Returns the operation mode of FAL. 0: IDLE state 1: Alignment mode 2: Synchronous acceleration mode 3: Active operation mode	
Application Example	uint_8 op_mode; op_mode = FAL_Get_op_mode();	

3.2.4.2 FAL_Get_Vout_Q14

Name of Function	FAL_Get_Vout_Q14	
Argument	void	
Return Value	uint16_t	Vout_Q14
Description	Returns the output voltage. $V_{out} = \sqrt{v_d^2 + v_q^2}$	
Application Example	int16_t Vout MAL_Vout_Q14 = FAL_Get_Vout_Q14();	

3.2.4.3 FAL_Get_Flag15Ts

Name of Function	FAL_Get_Flag15Ts	
Argument	void	
Return Value	uint8_t	0 or 1
Description	Returns the state of the 1.5 Ts delay compensation flag of SVPWM. 0: Disabled 1: Enabled	
Application Example	<pre> if (FAL_Get_Flag15Ts() == 0) { if (Spd_cmd_rpm >= 3000) { FAL_Set_Flag15Ts(1); } } else { if (Spd_cmd_rpm < 2500) { FAL_Set_Flag15Ts(0); } } </pre>	

3.2.4.4 FAL_Get_Tsamp_NUM

Name of Function	FAL_Get_Para_C15Ts_Qx	
Argument	uint16_t	1 or 2 1 returns Tsamp_1. 2 returns Tsamp_2.
Return Value	uint16_t	Tsamp_Num
Description	For single-shunt current detection, two different ADC sampling points (automatically calculated by the FAL_set_single_shunt function) must be configured at every PWM cycle. The FAL_Get_Tsamp_NUM function returns sampling points of the single shunt.	
Application Example	<pre>//In Top Interrupt Service Routine MP0->ATR1 = (1 << 16) ((FAL_Get_Tsamp_NUM(1)) & 0xFFFF); MP0->ATR2 = (1 << 16) ((FAL_Get_Tsamp_NUM(2)) & 0xFFFF);</pre>	

3.2.4.5 FAL_Get_Acc_Theta_Q14

Name of Function	FAL_Get_Acc_Theta_Q14	
Argument	int16_t*	Assumes the Q14 form, and falls in the range of -16384 to 16384. -16384 corresponds to $-\pi$, and 16384 to π .
Return Value	void	
Description	Returns the synchronous acceleration angle θ_{acc} .	
Application Example	<pre>if (FAL_Get_op_mode() == 2) { FAL_Get_Acc_Theta_Q14(&MAL_Theta_Q14); } else { MAL_Theta_Q14 = MAL_Est_Theta_Q14; }</pre>	

3.2.4.6 FAL_Get_Target_C15Ts_Qx

Name of Function	FAL_Get_Target_C15Ts_Qx	
Argument	void	
Return Value	uint16_t	C15Ts_Target_Qx
Description	Returns the constant to be used in time delay compensation for digital PWM. The constant is calculated by aFDT and stored as an array element, and can be retrieved using the FAL_Get_Target_C15Ts_Qx function. PWM time delay compensation is required for high speed motor operations, but a sudden introduction of PWM time delay compensation may cause a rapid change in the commanded voltage, thus producing pulsating currents. When starting to compensate for PWM time delay, therefore, pulsating currents may need to be reduced by retrieving the current control constant and gradually increasing it to reach the target value (use the FAL_Set_C15Ts_Qx function to apply the time delay compensation constant within the algorithm).	
Application Example	<pre> int16_t Spd_cmd_rpm //Commanded speed variable int16_t MAL_C15Ts_Qx; //Time delay compensation constant if (FAL_Get_Flag15Ts() == 0) { if (Spd_cmd_rpm >= 3000) { FAL_Set_Flag15Ts(1); } } else { if (Spd_cmd_rpm < 2500) { FAL_Set_Flag15Ts(0); } } if(FAL_Get_Flag15Ts()) { if (MAL_C15Ts_Qx < FAL_Get_Target_C15Ts_Qx()) { MAL_C15Ts_Qx++; } } else { if (MAL_C15Ts_Qx > 0) { MAL_C15Ts_Qx--; } } FAL_Set_C15Ts_Qx(MAL_C15Ts_Qx); //Time delay compensation constant applied //within algorithm using this function </pre>	

Chapter 4. MAL (Motor Application Layer)

4.1 Overview

The MAL (Motor Application Layer) eventually operates the motor using the functions of the Motor Driver Layer and FOC Algorithm Layer.

Users are recommended to avoid writing MCU-dependent codes. The sequences of the Motor Driver Layer and FOC Algorithm Layer functions in MAL must be kept to ensure normal functionality.

Codes independent of the sequences may be modified to fit application requirements.

4.2 Description of Functions

This section describes the basic functions of MAL.

For more details, refer to the source codes of ABOV 1-Shunt FOC Library.

4.2.1 MAL_init

Name of Function	MAL_init	
Argument	void	
Return Value	void	
Description	This function initializes the FOC library. It is called when first running the library. The code may be modified to fit application requirements.	

4.2.2 MAL_MPWM_Handler

Name of Function	MAL_MPWM_Handler	
Argument	void	
Return Value	void	
Description	This function is called by MPWMn_IRQHandler of MDL, and processes the MPWM interrupt handler. It is the most important function in the library, and uses the functions of both MDL and FAL. The code may be modified to fit application requirements while maintaining the sequence in the provided example is changed.	

4.2.3 MAL_mainloop

Name of Function	MAL_mainloop	
Argument	void	
Return Value	void	
Description	The main loop function repeats infinitely. It is used to control the motor operation process. The code may be modified to fit application requirements.	

Chapter 5. aFDT (ABOV FOC Development Tool)

5.1 Overview

This parameter configuration software can be used for the ABOV 1-Shunt FOC Library.

When parameters are configured for each block using aFDT, data arrays of 16-bit half words are generated and then transferred to the FAL (FOC Algorithm Layer), therein set as FAL internal parameters.

aDFT uses UART when transmitting these parameter arrays to FAL.

This data then can be exported as a C file. This C file includes data arrays that configure parameters. The parameters are successfully configured once the C file is compiled with other configuration files and the compilation is downloaded.

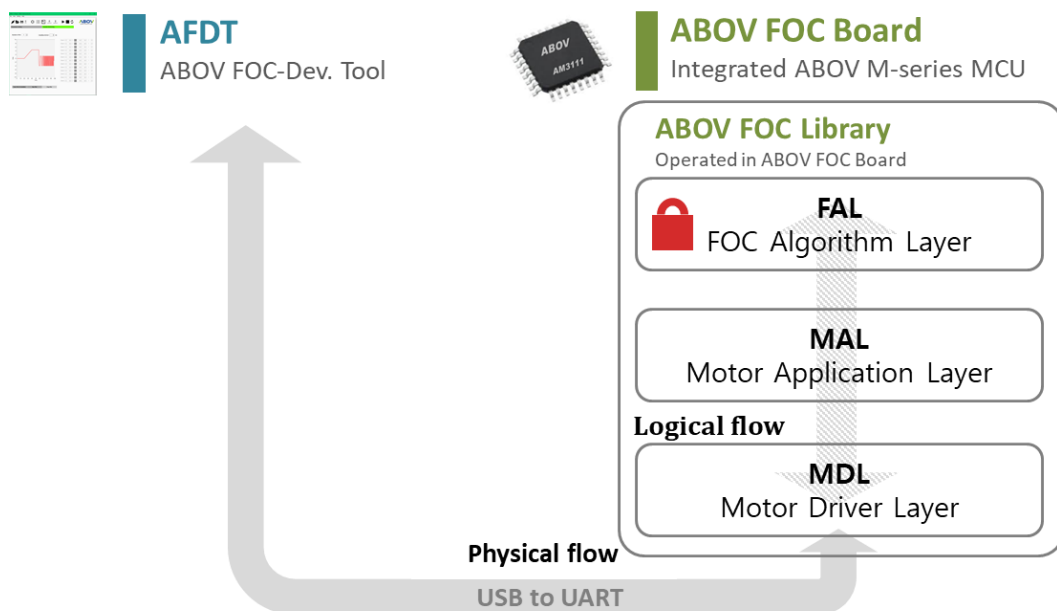


Figure 5.1 Block diagram of aFDT communications

5.2 1-Shunt / 3-Shunt Selection

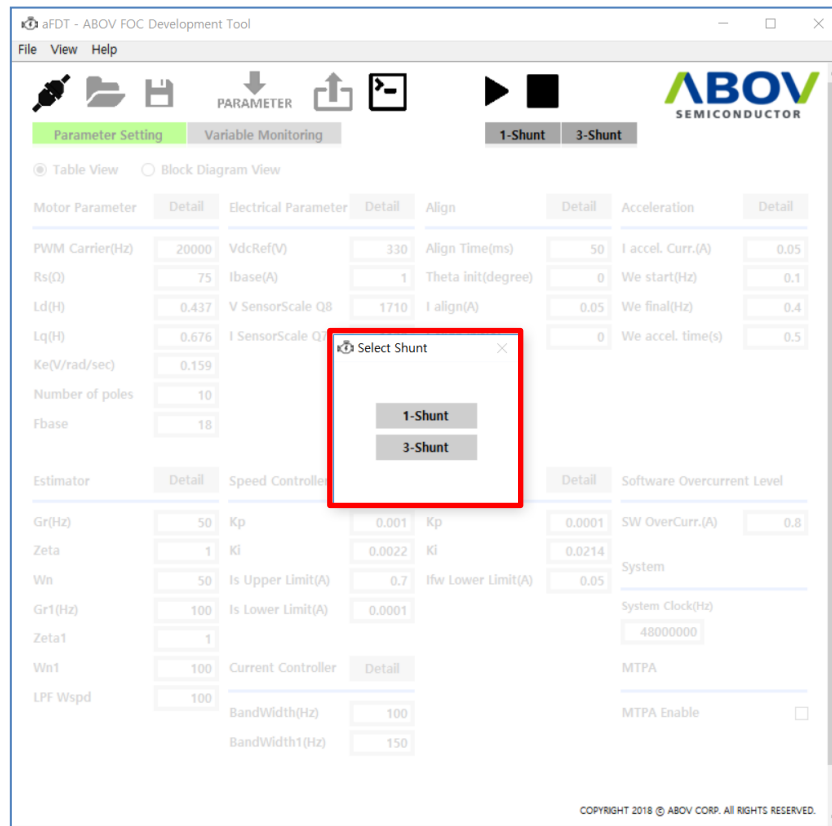


Figure 5.2 Selection page of FOC algorithms

Selecting the 'Enable' icon brings up a screen where you can select 1-Shunt and 3-Shunt as shown in Figure 5.3. You can choose according to the Algorithm used by the application.

5.3 Parameter Configuration

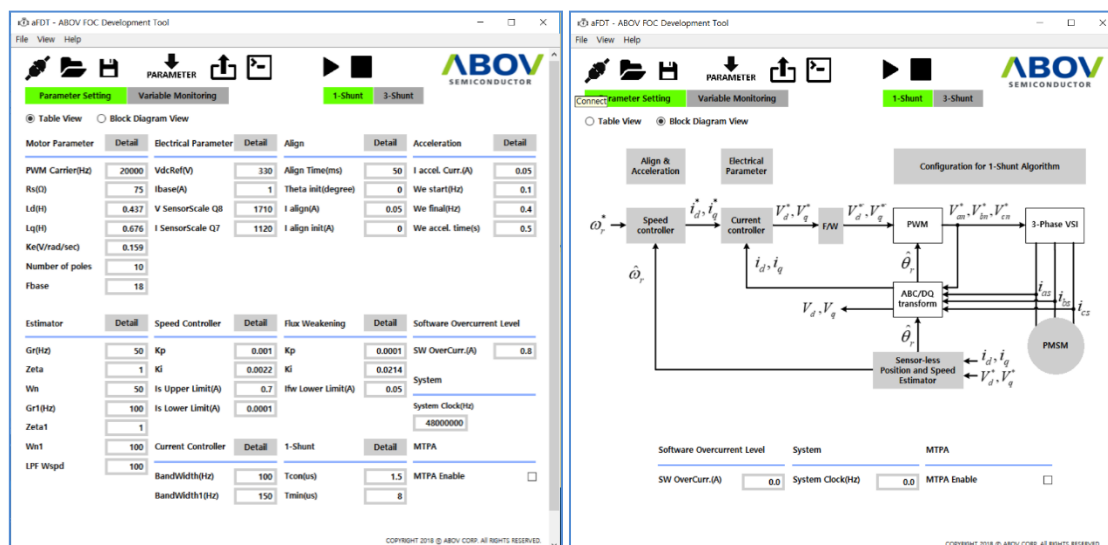


Figure 5.3 Main page of the parameter generation

Values are set for each block according to motor characteristics. Parameter configuration can be performed either in Table View or in Block Diagram View as preferred by the user: Table View lacks details but offers a good overview of all parameters, while Block Diagram View enhances understanding of the working structure of the algorithm using a block diagram. Clicking the Detail button for a block pops up a separate configuration window containing details. Further details on each parameter can be found in the section titled Parameter Description.



After configuration, clicking the **PARAMETER** button sends the parameters to the ABOV FOC Platform Board (aFPB). The internal parameters of the library can be modified and updated in real time. **Note that the motor must not be running when sending the parameters.**

The transferred parameters are not stored permanently, but reset at MCU reset. For permanent storage, the parameters must be exported, such that the project includes the parameter arrays. Further details on Export can be found in the section titled Export.

5.4 Save/Load

5.4.1 Save

Click  to display the following window:

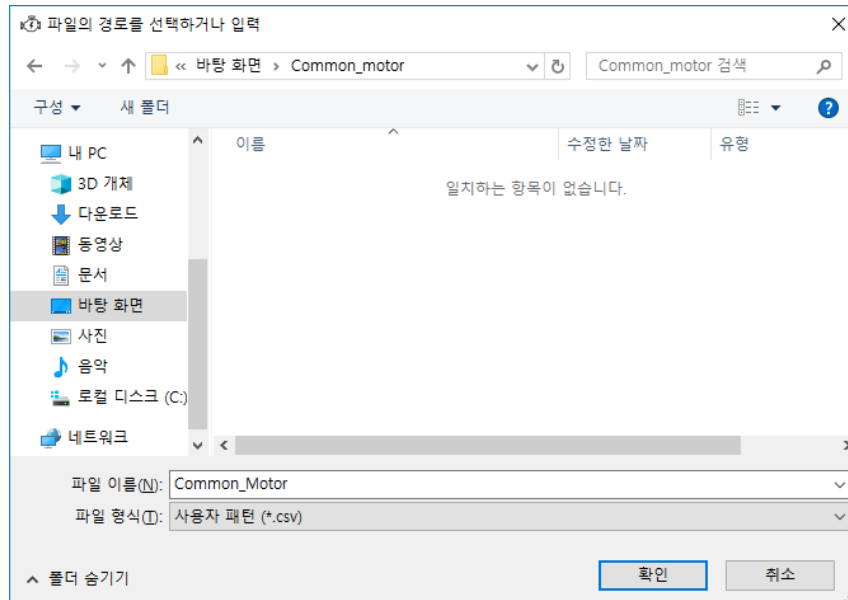


Figure 5.4 Popup for Save in aFDT

Set the file name, and click OK. The file is saved as a CSV file, and the stored values can be checked without reading with aFDT. The file can be opened in Microsoft Excel. When opened in Notepad, the following is displayed:

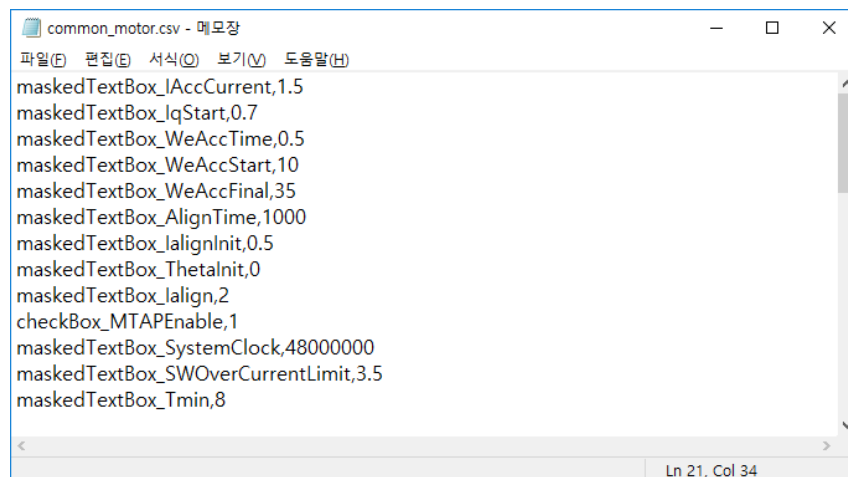


Figure 5.5 CSV file opened in Notepad

If the CSV file is corrupt or invalid, open it in Notepad to check for errors.

5.4.2 Load

Click  to display the following window:

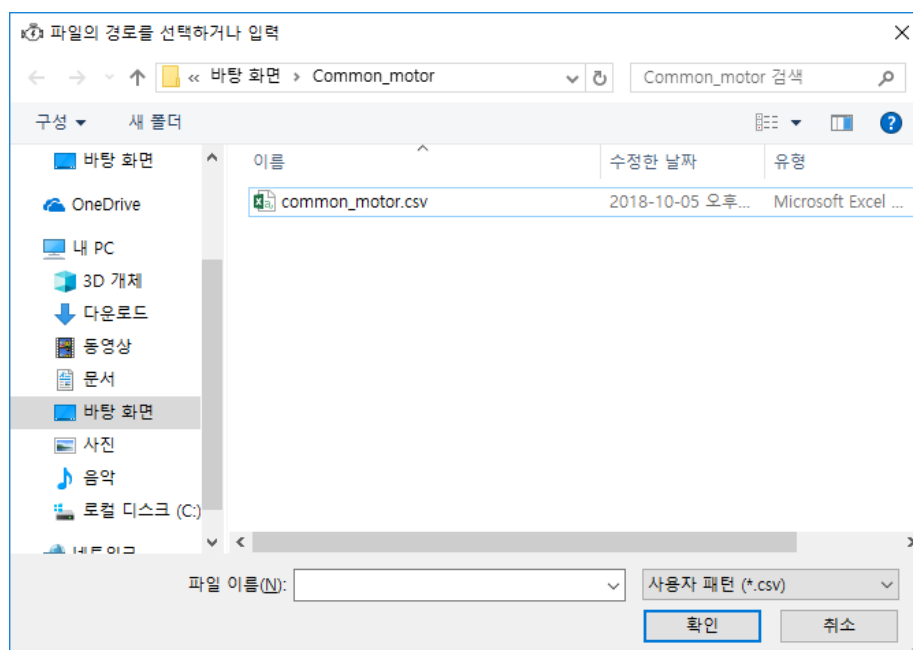


Figure 5.6 Popup for parameter generator loading

When the selected CSV file is read, the stored values are loaded on aFDT.

5.5 Export

Click  to display the following window:

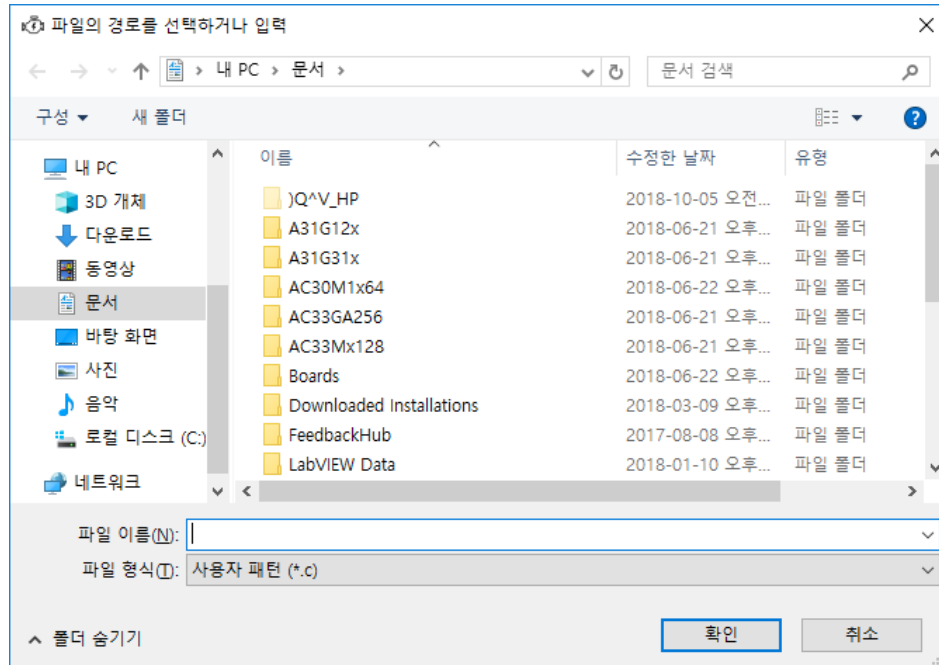


Figure 5.7 Popup for Export in aFDT <1>

Go to the installation path of the ABOV 1-Shunt FOC Library.

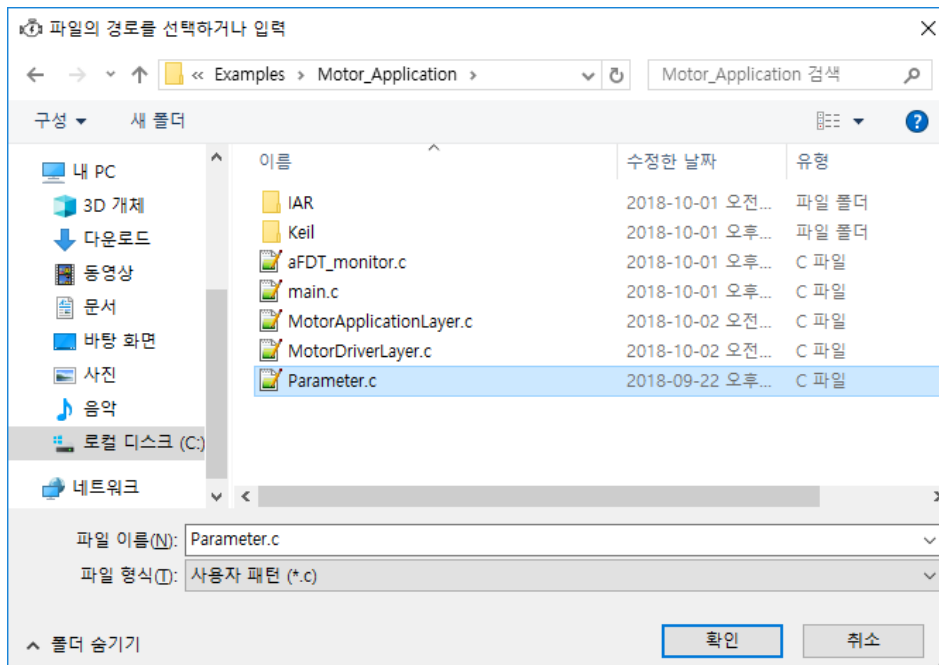


Figure 5.8 Popup for Load in aFDT <2>

Select Parameter.c, and click Save. The previous values in Parameter.c are replaced by new values.

The compiler then applies those new settings when it is compiling. Once the compilation is downloaded, motor control is enabled according to the aFDT settings.

The header of a C file export contains the following information:

```

/*****
*@file.....Parameter.C
*@brief.....File of Parameter data
*@version.....0.0.5 (aFDT)
*@algorithm...1-Shunt
*@date.....2019-07-15 12:20:10
*@author.....ABOV-AE
*
*Copyright (C) 2019, ABOV Semiconductor
*All rights reserved.
*
*****/

```

Figure 5.9 Header of a C file export

Table 5.1 Items in the header of a C file export

Item	Description
@File	Records the name of the stored file.
@brief	Describes the file.
@version	Records the version of aFDT that created the file.
@date	Records the date on which the file was created by aFDT.
@author	Records the name of the ABOV team that created the file.

5.6 Parameter Description

This section describes aFDT parameters for each Detail window.

5.6.1 Motor Parameter

Includes integer values required for motor operation, such as resistance and inductance.

Motor Parameter

Rs : Stator Resistance

Ld : Daxis Inductance

Lq : Qaxis Inductance

Ke : Back-EMF Constant

F_{base} : Maximum mechanical operating speed

PWM Carrier(Hz) 20000

Rs(Ω) 75

Ld(H) 0.437

Lq(H) 0.676

Ke(V/rad/sec.) 0.159

Number of poles 10

Fbase 18

OK Cancel

Figure 5.10 Motor parameters

Table 5.2 Motor Parameter

Item	Description
PWM Carrier(Hz)	Switching frequency
Rs(Ohm)	Winding resistance of motor (phase resistance)
Ld(H)	Inductance along magnetic flux axis
Lq(H)	Inductance along torque axis
Ke(V/rad/sec)	Back-Electromotive Force Constants
Number of poles	Number of poles
Fbase(Hz)	Maximum mechanical speed

5.6.2 Electrical Parameter

Includes values required for inverter operation.

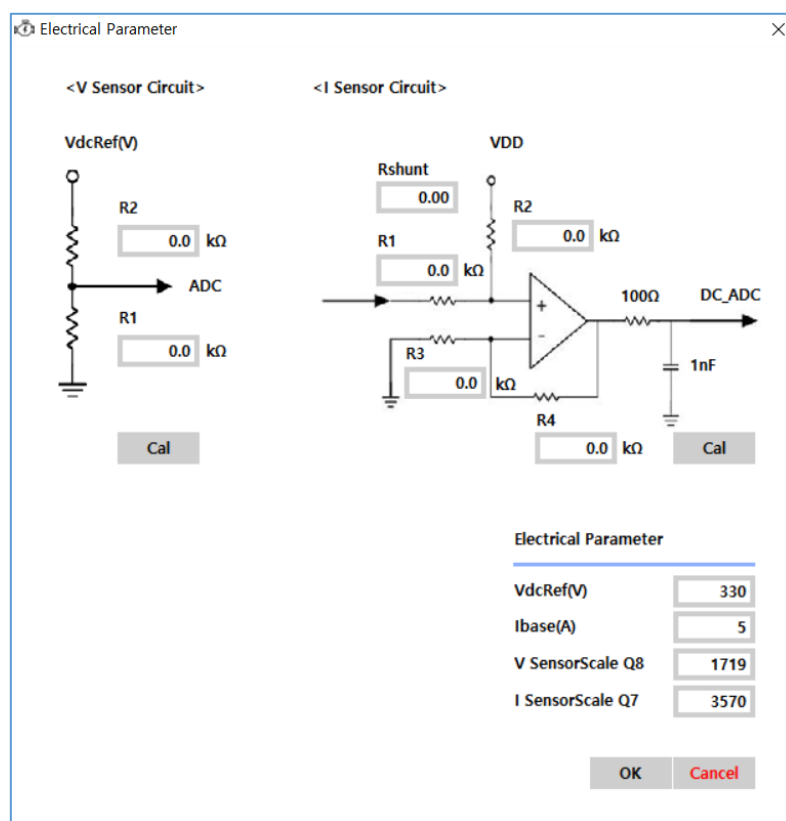


Figure 5.11 Electrical parameters

Table 5.3 Electrical Parameter

Item	Description
VdcRef(V)	Inverter DC_link voltage: Used to calculate voltage scale
Ibase(A)	Maximum current: Used to calculate current scale
V SensorScale	Voltage scale: Used to calculate sensed voltage
I SensorScale	Current scale: Used to calculate sensed current

5.6.3 Align

The section of forced alignment of the rotor to the d-axis by means of a current flow.

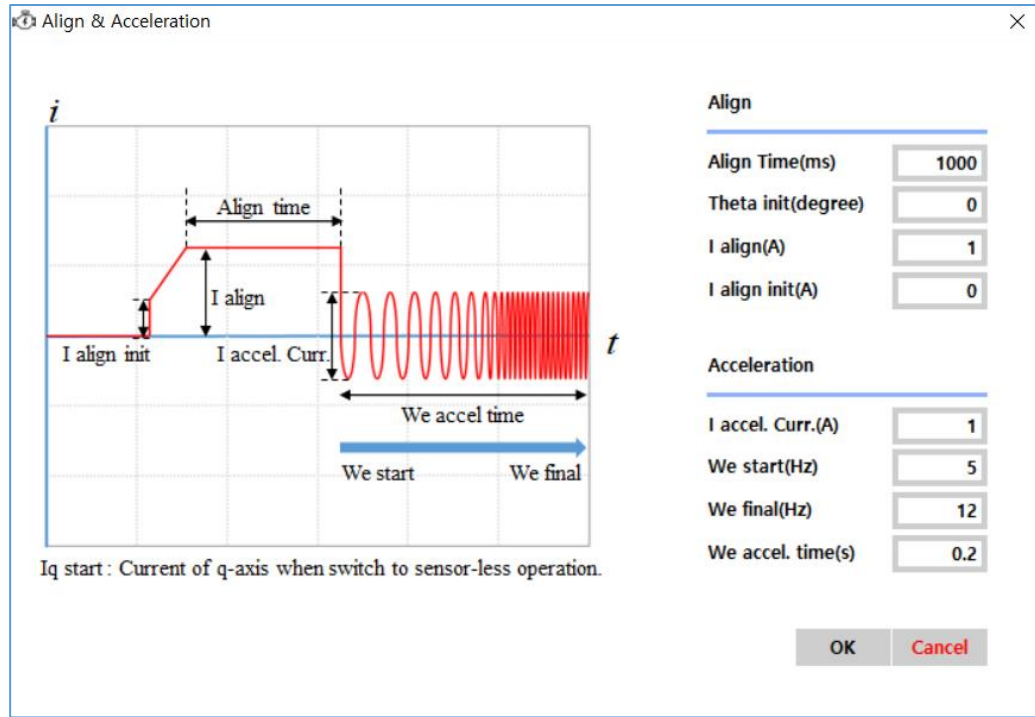


Figure 5.12 Align parameters

Table 5.4 Align

Item	Description
AlignTime(ms)	Forced alignment time: The time at which the rotor maintains its alignment position at the specified initial angle
Theta init(degree)	Initial angle of forced alignment: Initial alignment angle of the rotor
I align(A)	Forced alignment current: Magnitude of the forced alignment current
I align init(A)	Initial current of forced alignment: Magnitude of the current at the start of forced alignment

5.6.4 Acceleration

The section of synchronous acceleration to detect Back-EMF.

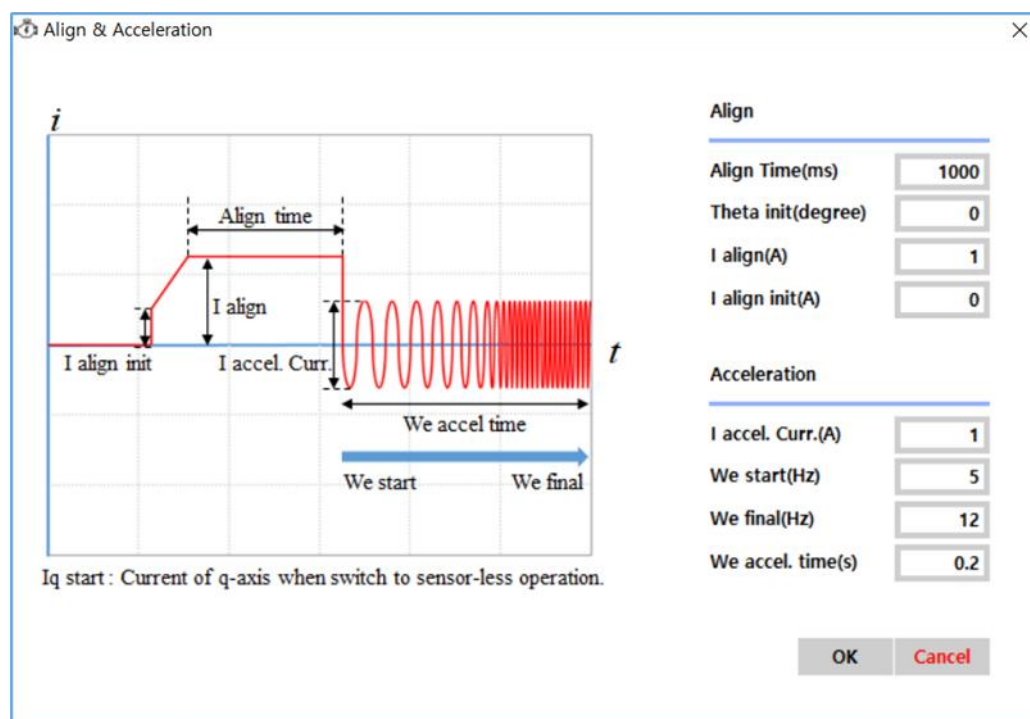


Figure 5.13 Acceleration parameters

Table 5.5 Acceleration

Item	Description
I accelation current(A)	Synchronous acceleration current: Current flowing during synchronous acceleration
We start(Hz)	Electrical angular velocity at start of synchronous acceleration
We final(Hz)	Electrical angular velocity at final of synchronous acceleration
We accelation time(ms)	Synchronous acceleration section: Time spent in synchronous acceleration section

5.6.5 Estimator

Estimates speed and position using Back-EMF.

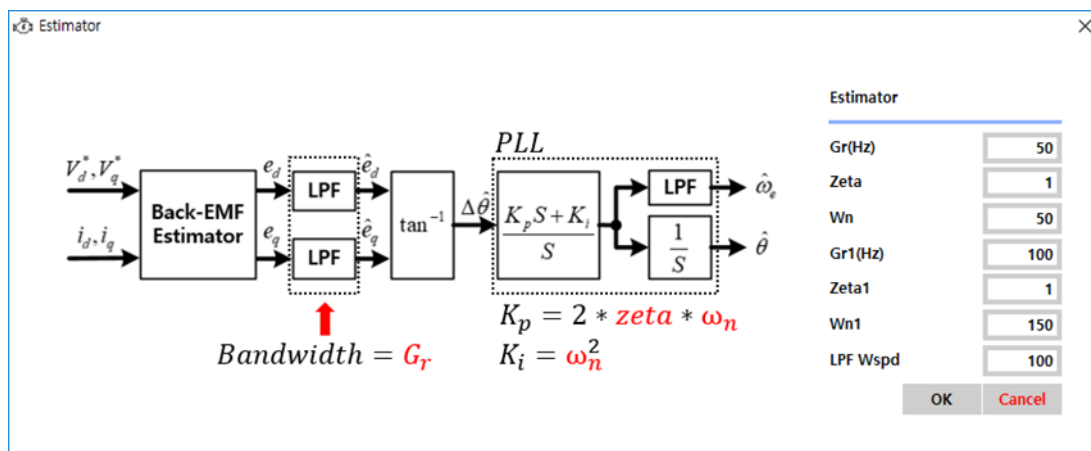


Figure 5.14 Estimator parameters

Table 5.6 Estimator

Item	Description
Gr(Hz)	Bandwidth of Back-EMF estimator
Zeta	Damping ratio of PLL
Wn	Natural frequency of PLL
Gr1(Hz)	Bandwidth of Back-EMF estimator
Zeta1	Damping ratio of PLL
Wn1	Natural frequency of PLL
LPF Wspd	Bandwidth of LPF

5.6.6 Speed Controller

Speed controller

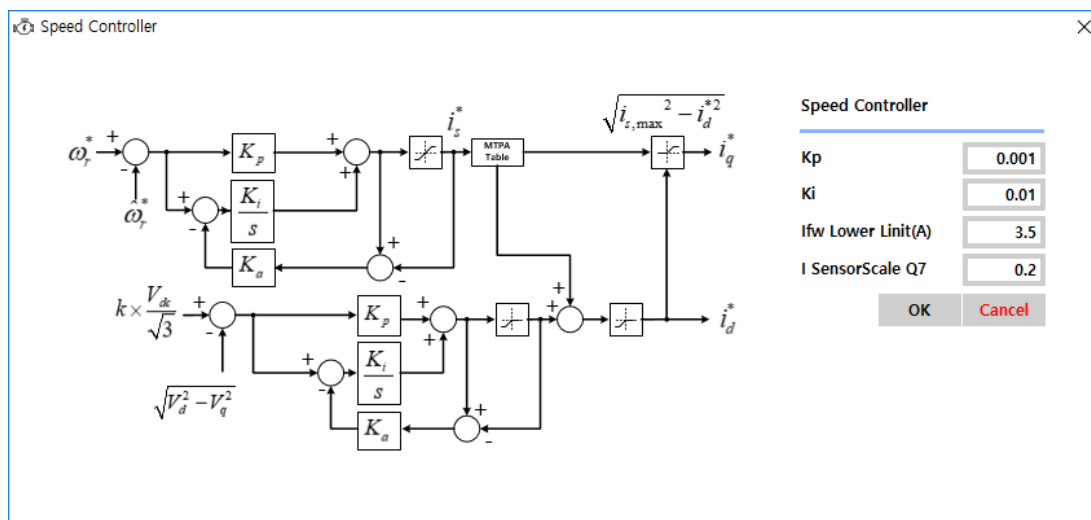


Figure 5.15 Speed controller parameters

Table 5.7 Speed Controller

Item	Description
Kp	Proportional gain
Ki	Integral gain
Is Upper Limit(A)	Maximum output limit of the speed controller
Is Lower Limit(A)	Minimum output limit of the speed controller

5.6.7 Current Controller

Current controller

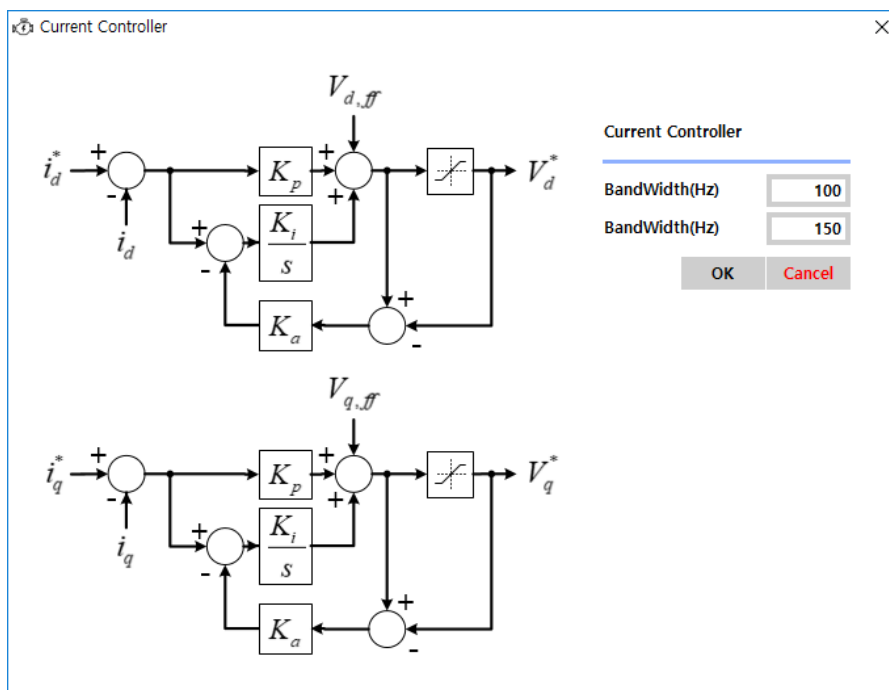


Figure 5.16 Current controller parameters

Table 5.8 Current Controller

Item	Description
Bandwidth(Hz)	Bandwidth of the current controller
Bandwidth1(Hz)	Bandwidth of the current controller

5.6.8 Flux Weakening

Flux weakening control

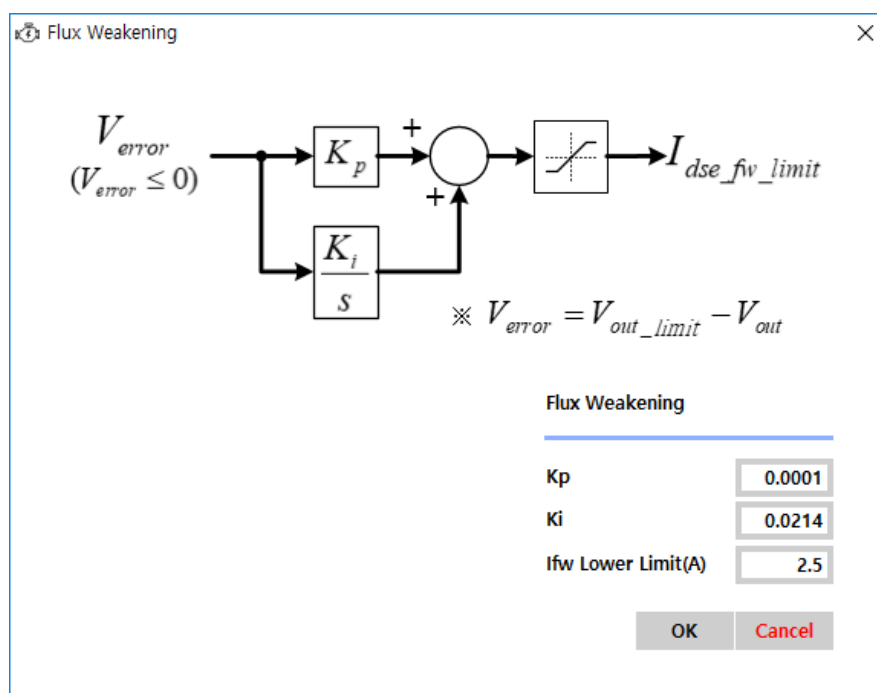


Figure 5.17 Flux weakening parameters

Table 5.9 Flux Weakening

Item	Description
Kp	Proportional gain
Ki	Integral gain
Ifw Lower Limit(A)	Output limit of the flux weakening controller

5.6.9 1-Shunt

Defines how to perform current feedback.

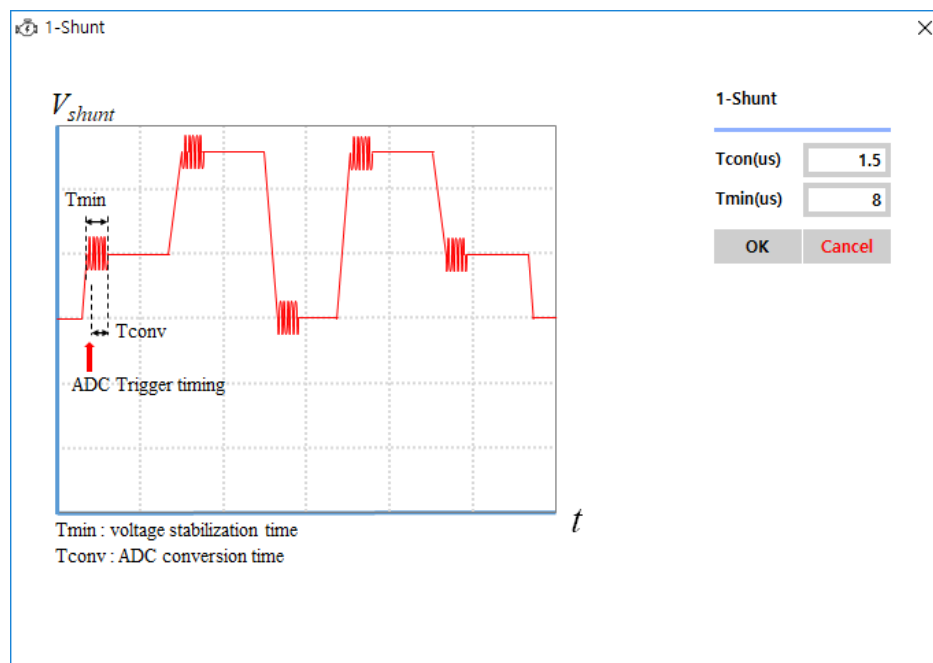


Figure 5.18 1-Shunt parameters

Table 5.10 1-Shunt

Item	Description
Tconv(us)	ADC conversion time
Tmin(us)	Minimum required time for current sensing

5.6.10 Software Overcurrent Level

Overcurrent protection level

Software Overcurrent Level

SW OverCurr.(A)

Figure 5.19 Software overcurrent level parameters

Table 5.11 Software Overcurrent Level

Item	Description
SW Overcurrent Level(A)	Software overcurrent fault level

5.6.11 System

MCU clock frequency

System

System Clock(Hz)

Figure 5.20 System parameters

Table 5.12 System

Item	Description
System Clock(Hz)	MCU system clock

5.7 Monitoring Function

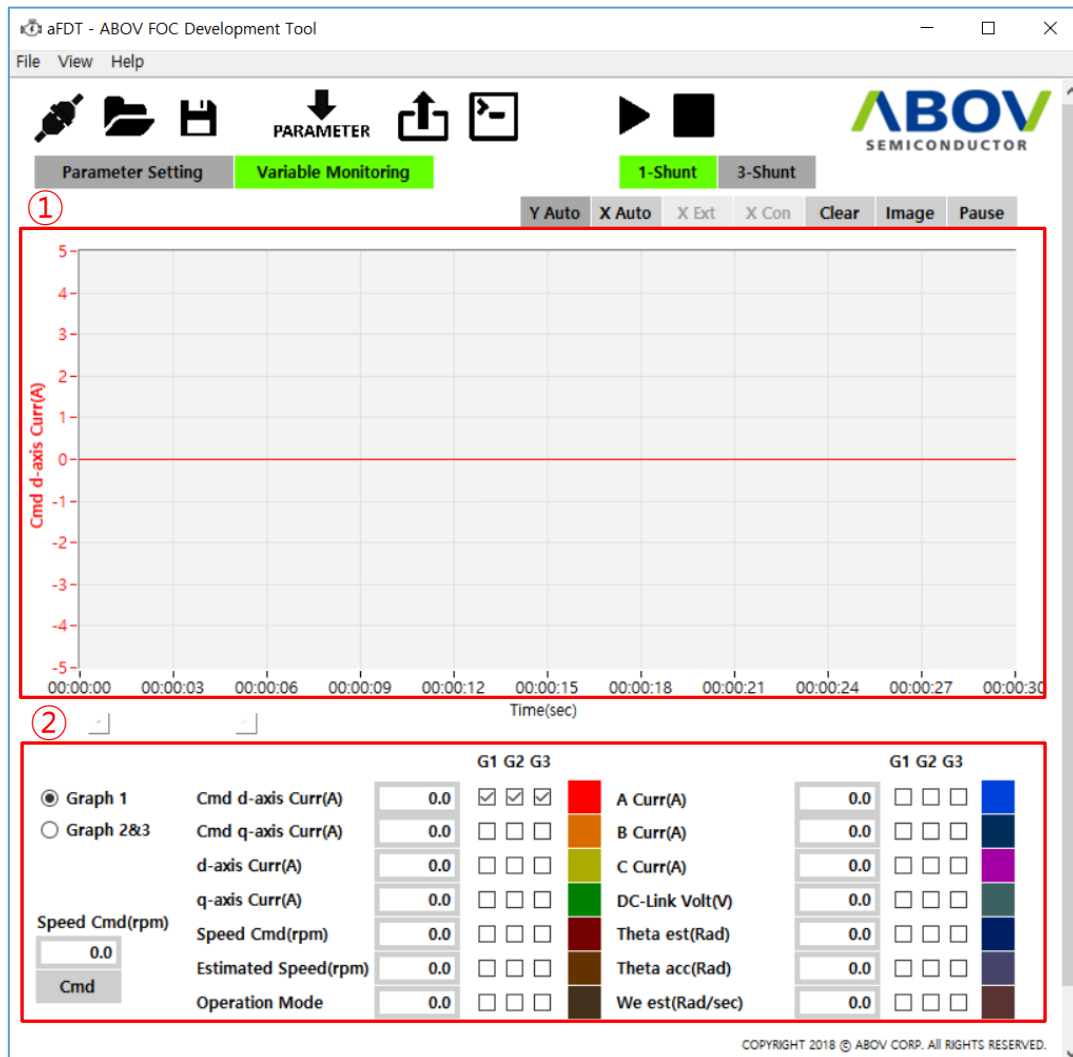


Figure 5.21 aFDT monitoring

aFDT monitoring enables the user to graphically monitor the 14 internal variables of the FOC Algorithm Layer (FAL).

- ① The graph area features a linear graph of the monitored data.
- ② The control area displays the retrieved data numerically, or enables entering a speed command for motor control.

5.7.1 Graph Area



Figure 5.22 Graph area

① These controls are used to set the graph display.

Table 5.13 Graph control buttons

Item	Description
Y Auto	Changes the Y scale automatically to match the maximum and minimum values of the Y-axis.
X Auto	Changes the X scale automatically to match the maximum value of the X-axis. When X Auto is enabled, the graph is displayed in the Clear state for 0–5 seconds; as time passes, the X scale changes in the order of 0–10 seconds, 0–20 seconds, and 0–30 seconds. The scale stops changing after 30 seconds, and the graph keeps displaying data for the last 30 seconds.
X Ext	Used to zoom in for the X-axis. The four levels available for zoom-in include 0–5 seconds, 0–10 seconds, 0–20 seconds, and 0–30 seconds. This button is enabled when X Auto is disabled.
X Con	Used to zoom out for the X-axis. The four levels available for zoom-in include 0–5 seconds, 0–10 seconds, 0–20 seconds, and 0–30 seconds. This button is enabled when X Auto is disabled.
Clear	Clears the data of the current graph.
Image	Saves the current graph as a JPG file.
Pause	Pauses the real-time graph display. Clicking this button again resumes the real-time display.

② This section shows the Y-axis data name and Y-axis scale. The minimum and maximum values may be modified. For the figure shown above, the minimum and maximum values can be changed by clicking on -5 or 5. The Y-scale will then be automatically adjusted.

5.7.2 Control Area

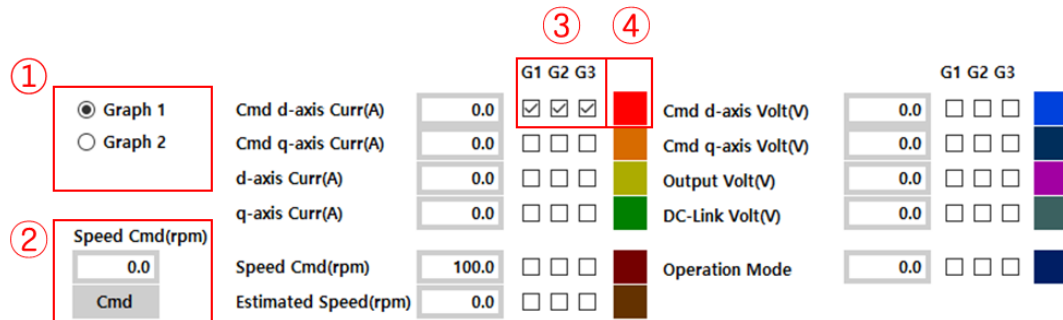
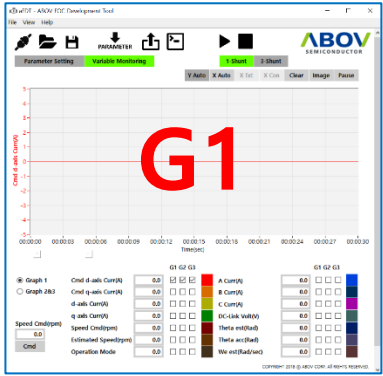
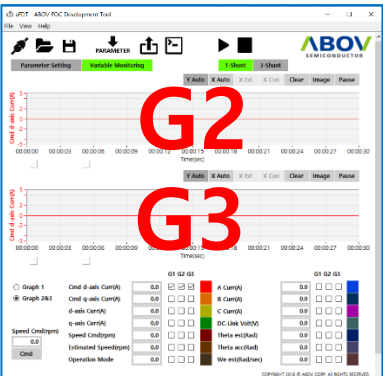


Figure 5.23 Graph area

- ① These are the radio buttons for graph display mode.

Table 5.14 Graph display mode

Item	Description
Graph 1	Displays G1. 
Graph 2 & 3	Displays G2 & G3. 

- ② Enter a speed control command. The motor will run at the commanded speed.
- ③ Choose whether to display each variable graph: G1, G2, and G3.
- ④ Choose a color for each variable. Click to display the color selection window. The variable's name and graph color will be updated accordingly in the graph area.